Συστήματα Παράλληλης Επεξεργασίας

Αναφορά Εργαστηριακών Ασκήσεων

Εργαστήριο Υπολογιστικών Συστημάτων
Σχολή Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών, ΕΜΠ

Χειμερινό εξάμηνο 2025-2026



Ομάδα parlab31

Γκίζης Κωστής - 03119013
Πλουμής Φίλιππος - 03120049
Ραφτόπουλος Μιχάλης - 03120114

ΠΙΝΑΚΑΣ ΠΕΡΙΕΧΟΜΕΝΩΝ

1 Εξοικείωση με το περιβάλλον προγραμματισμού **3**
1.1 Conway's Game of Life 3
2 Παραλληλοποίηση και βελτιστοποίηση αλγορίθμων σε αρχιτεκτονικές κοινής μνήμης **6**
2.1 Παραλληλοποίηση και βελτιστοποίηση του αλγορίθμου K-means 6
2.1.1 Shared clusters 6
2.1.2 Copied clusters and reduce 11
2.1.3 Αμοιβαίος Αποκλεισμός - Κλειδώματα 16
2.2 Παραλληλοποίηση του αλγορίθμου Floyd-Warshall 20
2.2.1 Ο αναδρομικός αλγόριθμος Floyd-Warshall 20
2.2.1.i) Εύρεση βέλτιστου B 22
2.2.1.ii) Εύρεση βέλτιστου CPU Affinity configuration 23
2.2.2 Ο Tiled αλγόριθμος Floyd-Warshall 25
2.2.2.i) Αποτελέσματα σε 1 NUMA node 27
2.2.2.ii) Σύγκριση ενός node με περισσότερα 28
2.2.2.iii) Τελικά συμπεράσματα για τη κλιμάκωση 28
2.2.3 Σύγκριση recursive και tiled FW 29
2.3 Ταυτόχρονες δομές δεδομένων 31
3 Παραλληλοποίηση και βελτιστοποίηση αλγορίθμων σε επεξεργαστές γραφικών **35**
3.1 K-Means Naive 35
3.2 K-Means Transpose 37
3.3 K-Means Shared 38
3.4 Bottleneck Analysis 39
3.5 K-Means All-GPU 41
4 Παραλληλοποίηση και βελτιστοποίηση αλγορίθμων σε αρχιτεκτονικές κατανεμημένης μνήμης **46**
4.1 Άλλη μια παραλληλοποίηση και βελτιστοποίηση του αλγορίθμου K-Means 46
4.2 Διάδοση θερμότητας σε δύο διαστάσεις 48
4.2.1 Μετρήσεις με έλεγχο σύγκλισης 48
4.2.2 Μετρήσεις χωρίς έλεγχο σύγκλισης 48

1 Εξοικείωση με το περιβάλλον προγραμματισμού

1.1 Conway's Game of Life

Σε αυτή την άσκηση μας ζητήθηκε να παραλληλοποιήσουμε κομμάτι κώδικα από το παιχνίδι Conway's Game of Life, με χρήση του προτύπου OpenMP. Στην υλοποίηση που μας δόθηκε προσθέσαμε το construct #pragma omp parallel for:

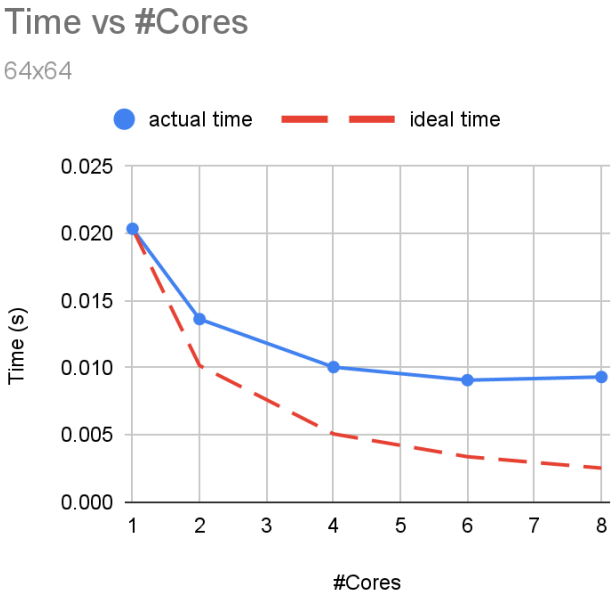
```
[...]

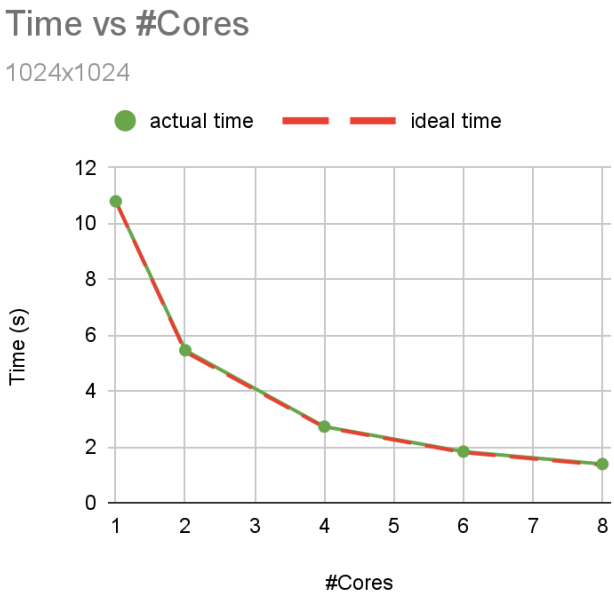
/*Game of Life*/

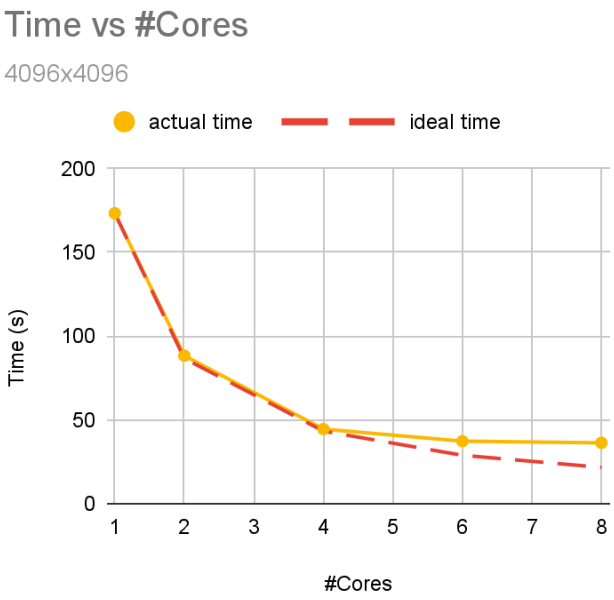
gettimeofday(&ts,NULL);
for ( t = 0 ; t < T ; t++ ) {
        #pragma omp parallel for private(j, nbrs)
        for ( i = 1 ; i < N-1 ; i++ )
                for ( j = 1 ; j < N-1 ; j++ ) {
[...]
```

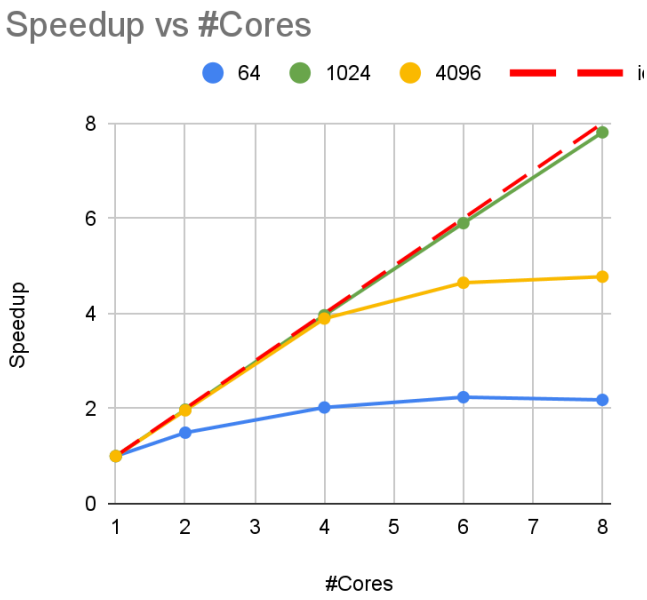
Αυτή η γραμμή δημιουργεί παράλληλες διεργασίες, στις οποίες διαμοιράζονται τα iterations του από κάτω for loop. Με το clause private(j, nbrs) το OpenMP δημιουργεί για κάθε εργάτη αντίγραφο των μεταβλητών j και nbrs.

Στη συνέχεια τρέξαμε το πρόγραμμα για πίνακες μεγέθους 64x64, 1024x1024 και 4096x4096 και καταγράψαμε τα παρακάτω αποτελέσματα για τον χρόνο και το speedup των εκτελέσεων σε πολλαπλούς πυρήνες.









Παρατηρούμε ότι για τα μεγέθη N=64 και 4096 το πρόγραμμα δεν κλιμακώνει καλά σε σχέση με το 1024 όπου επιτυγχάνεται σχεδόν ιδανική επιτάχυνση. Πιο συγκεκριμένα για N=64 η απόκλιση αυτή φαίνεται από την εισαγωγή μόλις ενός επιπλέον πυρήνα, ενώ για N=4096 η επιτάχυνση χειροτερεύει μετά τους 4 πυρήνες.

Οι παρατηρήσεις μας μπορούν να εξηγηθούν από δύο διαφορετικούς παράγοντες που περιορίζουν το speedup: το κόστος παραλληλοποίησης και οι προσβάσεις στη μνήμη. Το κόστος παραλληλοποίησης

οφείλεται στον συντονισμό των παράλληλων προγραμμάτων, εξαρτάται από το πλήθος των πυρήνων και είναι σταθερό για τα διάφορα μεγέθη της εισόδου. Οι προσβάσεις στην μνήμη αποτελούν καθυστέρηση καθώς, ενώ οι υπολογιστικοί πυρήνες είναι πολλοί, η μνήμη και το data bus είναι μονά και απαιτούν σειριακή πρόσβαση. Το πρόβλημα της μνήμης είναι ιδιαίτερα αισθητό στην περίπτωσή μας, καθώς για κάθε υπολογισμό έχουμε μεγάλο αριθμό αναγνώσεων (memory bandwidth bound problem).

Στην περίπτωση του πίνακα 64x64, οι προσβάσεις στην μνήμη είναι λίγες και επομένως δεν αποτελούν πρόβλημα. Επειδή όμως και το υπολογιστικό κομμάτι είναι μικρό, το overhead της παραλληλοποίησης δεν είναι καθόλου αμελητέο και λαμβάνει καθοριστικό ρόλο στην μείωση της κλιμακωσιμότητας. Αντίθετα, στην περίπτωση του πίνακα 4096x4096 έχουμε μεγάλο υπολογιστικό μέρος και άρα αμελητέο overhead. Ωστόσο, το μέγεθος των πινάκων δημιουργεί ανάγκη για πολλαπλές προσβάσεις στη μνήμη κι έτσι έχουμε πάλι μείωση της επίδοσης. Στη περίπτωση του 1024x1024 έχουμε τη χρυσή τομή των δύο παραμέτρων: σημαντικά μεγάλο υπολογιστικό κομμάτι, ώστε να μην μας επηρεάζει πολύ το κόστος παραλληλοποίησης αλλά και κατάλληλο μέγεθος πινάκων ώστε να χωράει στη κρυφή μνήμη.

2 Παραλληλοποίηση και βελτιστοποίηση αλγορίθμων σε αρχιτεκτονικές κοινής μνήμης

2.1 Παραλληλοποίηση και βελτιστοποίηση του αλγορίθμου K-means

Ο στόχος της παρούσας άσκησης είναι η παραλληλοποίηση του γνωστού αλγορίθμου συσταδοποίησης K-means. Για τον σκοπό αυτό υλοποιήθηκαν και συγκρίθηκαν δύο προσεγγίσεις: στην πρώτη επιβλήθηκε κοινόχρηστη μνήμη μεταξύ των threads, ενώ στη δεύτερη δημιουργήθηκαν και συγχωνεύθηκαν κατάλληλα αντίγραφα του πίνακα που αποθηκεύει τις πληροφορίες των clusters.

2.1.1 Shared clusters

Κατά την πρώτη μας προσέγγιση, αυτή της κοινής μνήμης, παραλληλοποιήσαμε με OpenMP τα κατάλληλα for loops, μέσω του `#pragma omp parallel for` directive. Μέσω του reduction`(+:delta)` ζητάμε από το OpenMP να κρατάει ένα αντίγραφο του `delta` για κάθε thread τα οποία θα προσθέτει μετά το πέρας της εκτέλεσής τους. Για να αποφευχθούν τα race conditions τους κοινούς πίνακες `newClusterSize` και `newClusters`, χρησιμοποιήσαμε το `#pragma omp atomic` directive, που διασφαλίζει ότι η εγγραφή στα αντίστοιχα cache blocks θα γίνει ατομικά.

```
[...]

#pragma omp parallel for private(index, j) reduction(+:delta)
for (i=0; i<numObjs; i++) {
    // find the array index of nearest cluster center
    index = find_nearest_cluster(numClusters, numCoords,
&objects[i*numCoords], clusters);

    // if membership changes, increase delta by 1
    if (membership[i] != index)
        delta += 1.0;

    // assign the membership to object i
    membership[i] = index;

    // update new cluster centers : sum of objects located within
#pragma omp atomic
    newClusterSize[index]++;
    for (j=0; j<numCoords; j++)
```

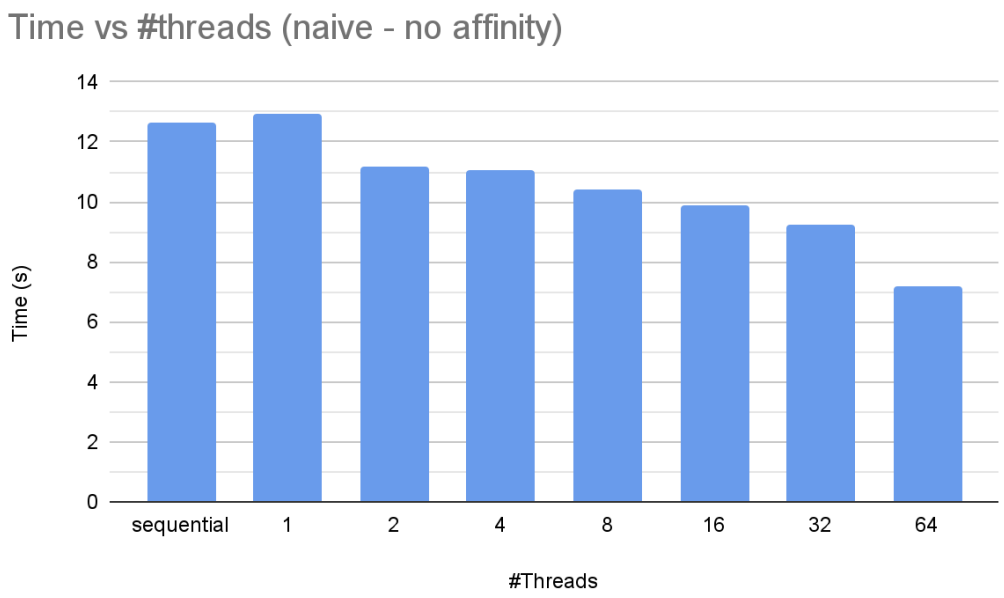
```
#pragma omp atomic
        newClusters[index*numCoords + j] += objects[i*numCoords + j];
}

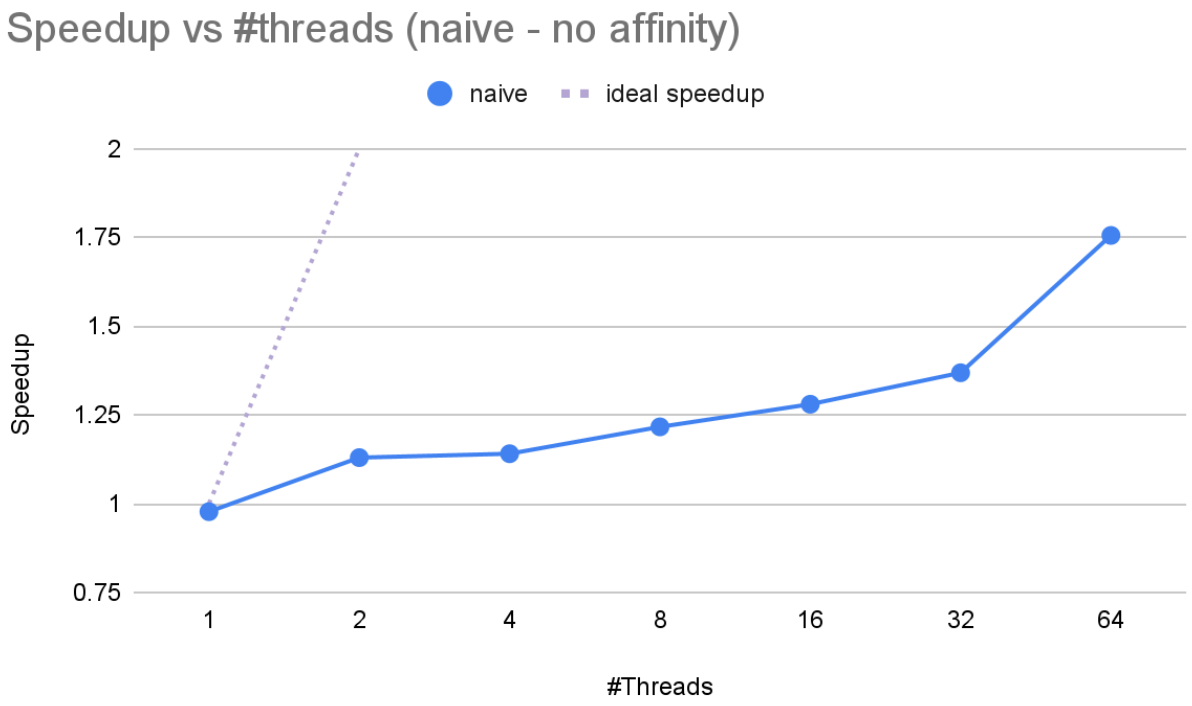
#pragma omp parallel for private(j)
// average the sum and replace old cluster centers with newClusters
for (i=0; i<numClusters; i++) {
    if (newClusterSize[i] > 0) {
        for (j=0; j<numCoords; j++) {
            clusters[i*numCoords + j] = newClusters[i*numCoords + j]
/ newClusterSize[i];
        }
    }
}

[...]
```

Μεταγλωττίσαμε το πρόγραμμα και το εκτελέσαμε με τις ζητούμενες παραμέτρους:
{Size, Coords, Clusters, Loops} = {256, 16, 32, 10}

Παρακάτω φαίνονται τα διαγράμματα χρόνου εκτέλεσης για 1, 2, 4, 8, 16, 32, 64 threads καθώς και του σειριακού αλγορίθμου και το αντίστοιχο speedup plot.'





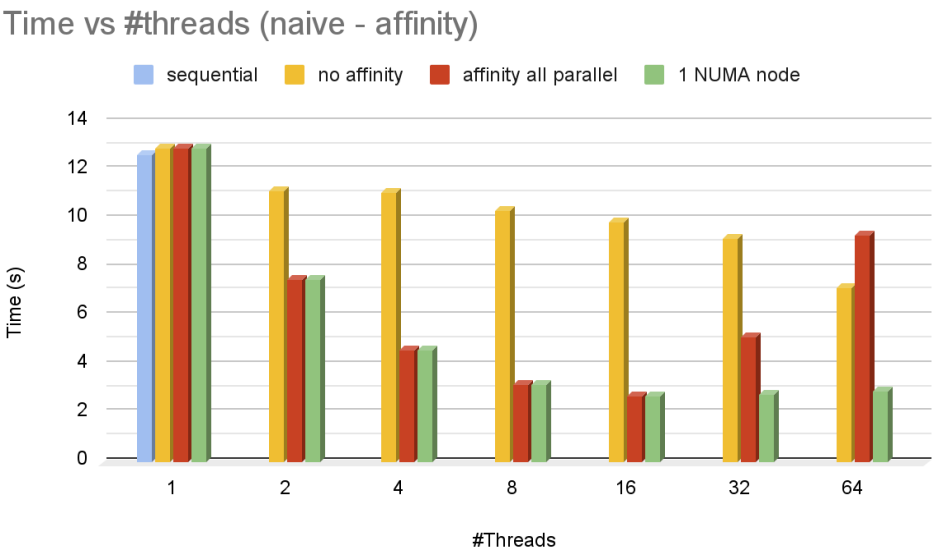
Σε αυτή την naive εκδοχή του αλγορίθμου, αν και η επεξεργασία των αντικειμένων γίνεται παράλληλα, οι ενημερώσεις των newClusters και newClusterSize γίνονται σε shared δεδομένα που προστατεύονται με atomic. Αυτό δημιουργεί έντονο synchronization bottleneck: όσο αυξάνεται ο αριθμός των threads, τόσο περισσότερα threads προσπαθούν να ενημερώσουν τα ίδια λίγα στοιχεία μνήμης. Συνολικά, η naive παράλληλη έκδοση είναι μεν πιο γρήγορη από την σειριακή αλλά το speedup που λαμβάνουμε είναι ελάχιστο, φτάνοντας μέχρι το 1.755.

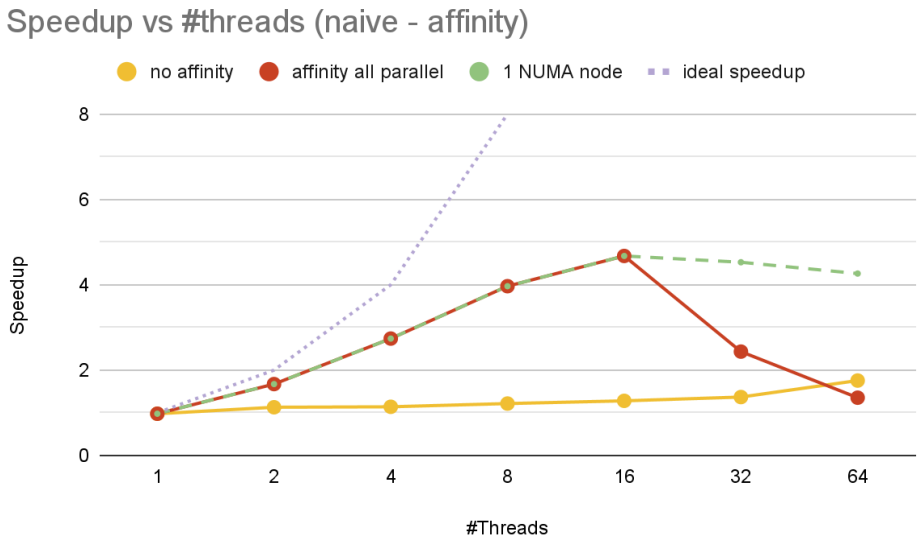
Μια ενδιαφέρουσα παρατήρηση είναι πως το speedup συνεχίζει να αυξάνεται και μετά τα 16 cores, αντίθετα ίσως από τη κοινή διαίσθηση. Αυτό μπορεί να εξηγηθεί αν αναλογιστούμε πώς το κάθε thread χρειάζεται να κάνει stalls - για οποιονδήποτε λόγο, είτε είναι memory requests, είτε κάποιο wait για το κρίσιμο κομμάτι του κώδικα. Σε αυτές τις περιπτώσεις λοιπόν, το να έχουμε παραπάνω από 16 threads είναι ωφέλιμο, διότι ο δρομολογητής μπορεί να βάλει άλλη διεργασία να τρέξει αυξάνοντας τη παραλληλοποίηση.

Μια επιπλέον παράμετρος των πειραμάτων μας αποτέλεσε το `GOMP_CPU_AFFINITY`. Κανονικά, ο scheduler του λειτουργικού συστήματος αποδεσμεύει και αναθέτει διαρκώς εκτελούμενα threads σε επεξεργαστικούς πυρήνες, ώστε τα προγράμματα να εμφανίζουν υψηλή αποκρισιμότητα. Αν και επιθυμητή σε περιβάλλον workstation, αυτή η λειτουργία είναι απαγορευτική στο πλαίσιο του HPC, αφού το overhead και το context switching επηρεάζουν σημαντικά την επίδοση της εκτέλεσης. Μέσω του `GOMP_CPU_AFFINITY` υποδεικνύουμε στον scheduler να αναθέτει τα threads σε

συγκεκριμένους πυρήνες και να εκτελούνται σε αυτούς μέχρι την ολοκλήρωσή τους. Εκτελέσαμε όλα μας τα πειράματα ξανά, με CPU Affinity, με σκοπό την μελέτη της επίδρασής της στην επίδοση.

Τα αποτελέσματα των πειραμάτων μας σε σύγκριση με την αρχική εκτέλεση φαίνονται στα παρακάτω διαγράμματα.





Από τα παραπάνω διαγράμματα παρατηρούμε τεράστια διαφορά στην απόδοση, όταν ορίζουμε τη παράμετρο `GOMP_CPU_AFFINITY`, δηλαδή όταν δεσμεύουμε κάθε thread σε ορισμένο CPU core.

Στη περίπτωση με το cpu affinity, το speedup αυξάνεται δραματικά. Η διαφορά αυτή οφείλεται στο γεγονός πως, εφόσον κάθε thread δεσμεύεται σε ένα cpu core και δεν μεταφέρεται από core σε core, είναι λιγότερο πιθανό να χάσει τα δεδομένα που χρειάζεται από την cache του, πράγμα το οποίο θα οδηγούσε σε καθυστερήσεις λόγω memory requests. Σε αυτή τη περίπτωση παρατηρούμε ότι το speedup μεγιστοποιείται πράγματι στα 16 threads. Αυτό δεν παραβαίνει ό,τι είπαμε προηγουμένως, διότι σε αυτή τη περίπτωση δεν υπάρχουν επιπλέον threads που μπορούν να τρέξουν στον πυρήνα όταν μια διεργασία κάνει stall, παρά μόνο 1 επιπλέον thread/core στα 32 threads και 3 επιπλέον threads/core στα 64.

Μια τελευταία παρατήρηση είναι πως παρόλο που χρησιμοποιούμε σχετικά μεγάλο αριθμό threads (16,32,64), δε παρατηρούμε ανάλογες επιδόσεις στο speedup, το οποίο φτάνει μόλις το 4.66. Η αιτία για αυτό είναι τα race conditions που αντιμετωπίζουν τα threads. Συγκεκριμένα, κάθε φορά που ένα thread χρειάζεται να γράψει στον πίνακα newClusterSize ή newCLusters πρέπει να το κάνει ατομικά και τα υπόλοιπα threads που επιθυμούν να γράψουν στον ίδιο πίνακα πρέπει να περιμένουν, όπως είπαμε και προηγουμένως.

Τέλος, η ομάδα μας επιθύμησε να διερευνήσει την επίδραση του αριθμού των NUMA nodes στην επίδοση της παράλληλης εκτέλεσης. Το ζήτημα αυτό έχει ενδιαφέρον, διότι με περισσότερους κόμβους έχουμε περισσότερους παράλληλους υπολογιστικούς πυρήνες, ταυτόχρονα όμως έχουμε να αντιγράψουμε και να συντηρήσουμε περισσότερα αντίγραφα των δεδομένων (αφού κάθε κόμβος έχει τη δική του μνήμη).

Για να προχωρήσουμε παρακάτω, είναι σημαντικό να κατανοήσουμε το μηχάνημα sandman στο οποίο δουλεύουμε. Πρόκειται για έναν πολυεπεξεργαστικό server αρχιτεκτονικής NUMA που περιλαμβάνει 4 φυσικούς επεξεργαστές Intel Xeon E5-4620 (Sandy Bridge) (sockets). Κάθε socket διαθέτει 8 φυσικούς πυρήνες και ενεργοποιημένο HyperThreading προσφέροντας 16 λογικούς πυρήνες, δίνοντας συνολικά 32 πυρήνες και 64 threads. Το Linux αναγνωρίζει 4 NUMA nodes, έναν για κάθε socket κατά αυτόν τον τρόπο:

- NUMA node 0: CPUs 0-7, 32-39
- NUMA node 1: CPUs 8-15, 40-47
- NUMA node 2: CPUs 16-23, 48-55
- NUMA node 3: CPUs 24-31, 56-63

Χρησιμοποιώντας την παραπάνω αρίθμηση, εκτελέσαμε πειράματα ισομοιράζοντας τα threads σε 1, 2 και 4 κόμβους (π.χ για 16 νήματα η `GOMP_CPU_AFFINITY="0-7,32-39"` δένει τα νήματα στον κόμβο 0, ενώ η `GOMP_CPU_AFFINITY="0-3,8-11,16-19,24-27"` σε όλους τους κόμβους αντίστοιχα κ.ο.κ). Παρατηρήσαμε λοιπόν ότι οι βέλτιστοι χρόνοι λήφθηκαν όταν τα νήματα περιορίστηκαν στους λιγότερους δυνατούς κόμβους, ενώ με την αύξηση των κόμβων ο χρόνος

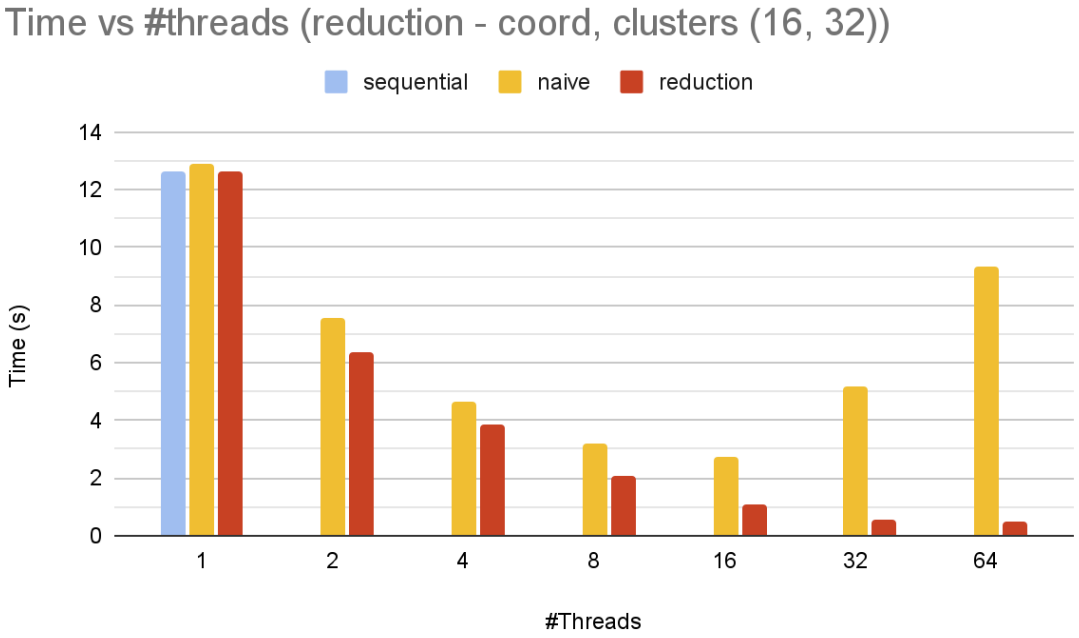
αυξανόταν σχεδόν γραμμικά. Αξίζει να σημειωθεί ότι για 32 και 64 νήματα ο χρόνος εκτέλεσης ήταν αρκετά χαμηλότερος όταν η εκτέλεση έγινε σε 1 κόμβο παρόλο που τα νήματα δεν χωράνε και "σπάει" η παραλληλοποίηση (πράσινο χρώμα στα διαγράμματα). Αυτή η παρατήρηση φανερώνει ότι, στη συγκεκριμένη εφαρμογή, ο παράλληλος υπολογισμός "πνίγεται" από τα shared writes. Το να σκορπίσουμε threads σε πολλά sockets επιδεινώνει το coherence traffic.

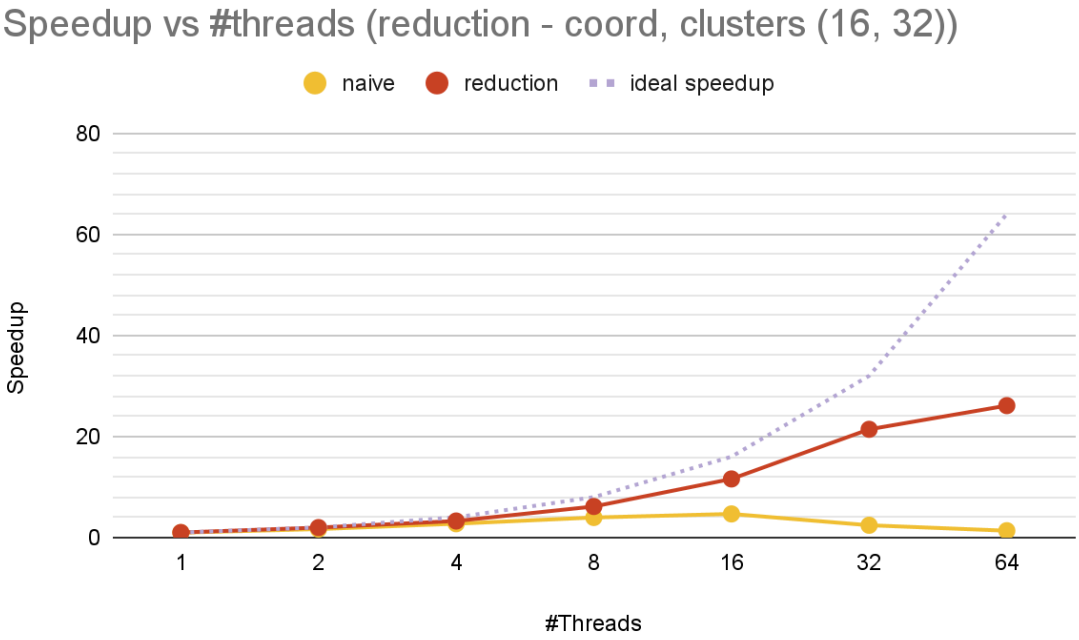
2.1.2 Copied clusters and reduce

Έπειτα προχωράμε σε μία δεύτερη εκδοχή του αλγορίθμου η οποία περιμένουμε να εμφανίσει καλύτερα αποτελέσματα. Η βασική της ιδέα είναι ότι αντί όλα τα threads να μοιράζονται και να ενημερώνουν κοινά τους πίνακες `newClusters[ ]` και `newClusterSize[]`, καθένα έχει ένα δικό του private αντίγραφο των πινάκων `local_newClusters[ ]` και `local_newClusterSize[]` οι οποίοι έχουν μία επιπλέον διάσταση με αριθμό ίσο με αυτόν του αριθμού των νημάτων. Η μεταβλητή delta αντιμετωπίστηκε με τον ίδιο τρόπο όπως προηγουμένως, ενώ τώρα μέσα στον παράλληλο βρόχο for κάθε νήμα ενημερώνει τον δικό του πίνακα. Αφού ολοκληρωθεί ο βρόχος το αρχικό master thread θα αναλάβει το reduction των τοπικών πινάκων στον τελικό κοινό.

Πλέον για όλες τις μετρήσεις χρησιμοποιήσαμε τη μεταβλητή περιβάλλοντος `GOMP_CPU_AFFINITY` καθώς έχουμε επιβεβαιώσει τη θετική επίδραση της. Ακολουθήσαμε την ίδια λογική με πριν και δοκιμάσαμε ξανά διαφορετικούς συνδυασμούς ανάθεσης νημάτων σε πυρήνες. Αυτή τη φορά όμως καλύτεροι χρόνοι επιτεύχθηκαν όταν τα νήματα "απλώθηκαν" σε περισσότερα NUMA nodes. Αυτό συμβαίνει διότι ουσιαστικά με αυτή την υλοποίηση καταπολεμούμε το πρόβλημα του συγχορισμού, αλλά το bottleneck μεταφέρεται στο memory bandwidth (memory-bandwidth-bound). Επομένως, όταν τα threads κατανέμονται σε πολλούς NUMA nodes, αυξάνεται το συνολικό διαθέσιμο bandwidth, γεγονός που μειώνει τον χρόνο εκτέλεσης. Για αυτό βλέπουμε ότι η απόδοση μετά τα 16 threads βελτιώνεται όταν τα threads κατανέμονται σε όλους τους NUMA nodes.

Παρακάτω φαίνονται τα διαγράμματα χρόνου εκτέλεσης και επιτάχυνσης για τις δύο παραλλαγές.



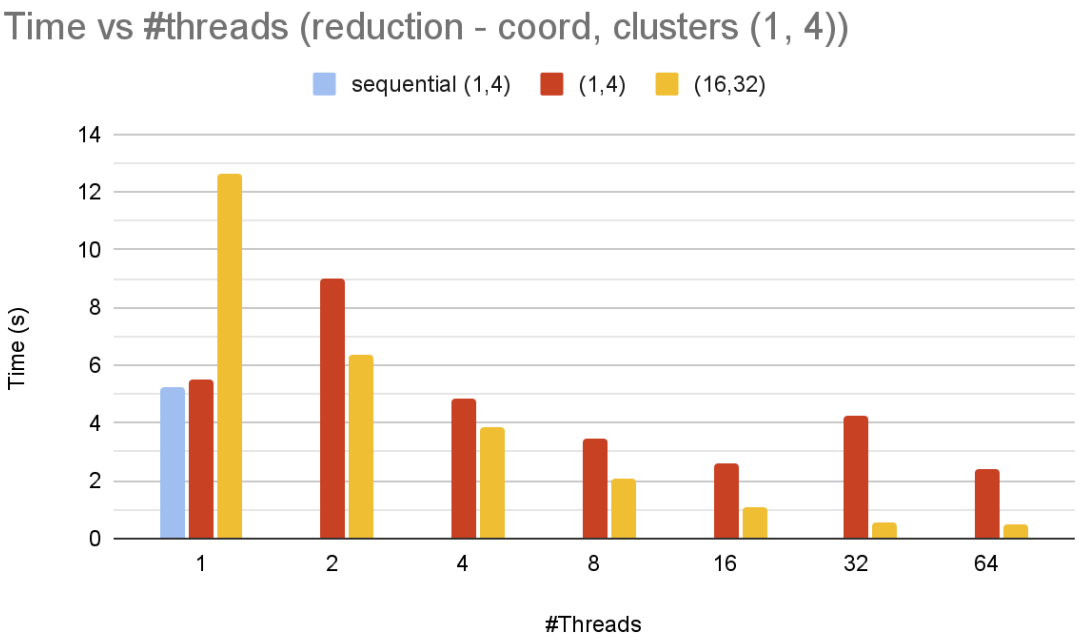


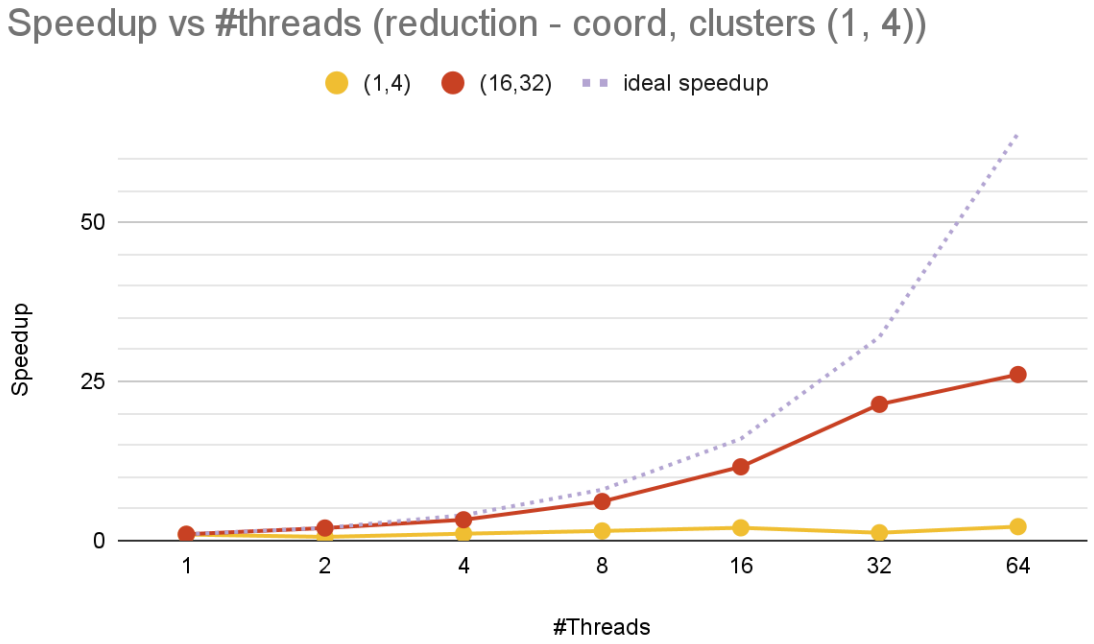
Παρατηρούμε πως τώρα η παραλληλοποίηση επιφέρει πολύ καλύτερες επιδόσεις σε σχέση με την αρχικη έκδοση καθώς ο χρόνος μειώνεται ομαλά με την αύξηση των νημάτων. Ειδικότερα, το πρόγραμμα εμφανίζει πολύ καλό scalability, αφού το speedup μέχρι τα 32 νήματα είναι πολύ κοντά

στο ιδανικό. Η μείωση της κλιμάκωσης που παρατηρείται από τα 32 στα 64 νήματα οφείλεται στην εισαγωγή του HyperThreading, δηλαδή οι 32 επιπλέον πυρήνες που χρησιμοποιούνται δεν είναι φυσικοί πυρήνες και δεν έχουν την ίδια συμβολή σε τέτοιου είδους προγράμματα.

Γενικότερα μπορούμε να συμπεράνουμε πως σε προγράμματα που απαιτούσαν πολλά locks κατά την εκτέλεση (frequent locking) το reduction κοστίζει λιγότερο, χωρίς όμως αυτό να να αποτελεί γενίκευση για την επιλογή shared ή copied και σε άλλου είδους περιπτώσεις.

Τέλος, γίνεται αλλαγή του configuration σε {Size, Coords, Clusters, Loops} = {256, 1, 4, 10} δηλαδή μειώνουμε τον αριθμό των συντεταγμένων σε 1 από 16 και τον αριθμό των clusters σε 4 από 32. Σε αυτή την περίπτωση καλύτεροι χρόνοι λήφθηκαν όταν μέσω του affinity περιορίσαμε τα νήματα σε όσο το δυνατόν λιγότερα NUMA nodes και τα αντίστοιχα διαγράμματα παρουσιάζονται παρακάτω.





Παρατηρούμε ότι η επίδοση πέφτει δραματικά και η κλιμάκωση είναι σαφώς χειρότερη, επιτυγχάνοντας μέγιστο speedup 2x στα 16 threads. Αρχικά το configuration αυτό είναι αρκετά πιο memory intensive (memory-bound) καθώς οι πίνακες είναι πολύ μικρότεροι και κάθε νήμα χρειάζεται να κάνει ελάχιστους υπολογισμούς πριν πραγματοποιήσει εγγραφή στη μνήμη. Το γεγονός αυτό όμως δεν αποτελεί τον μοναδικό λόγο στον οποίο οφείλεται αυτή η επίδοση.

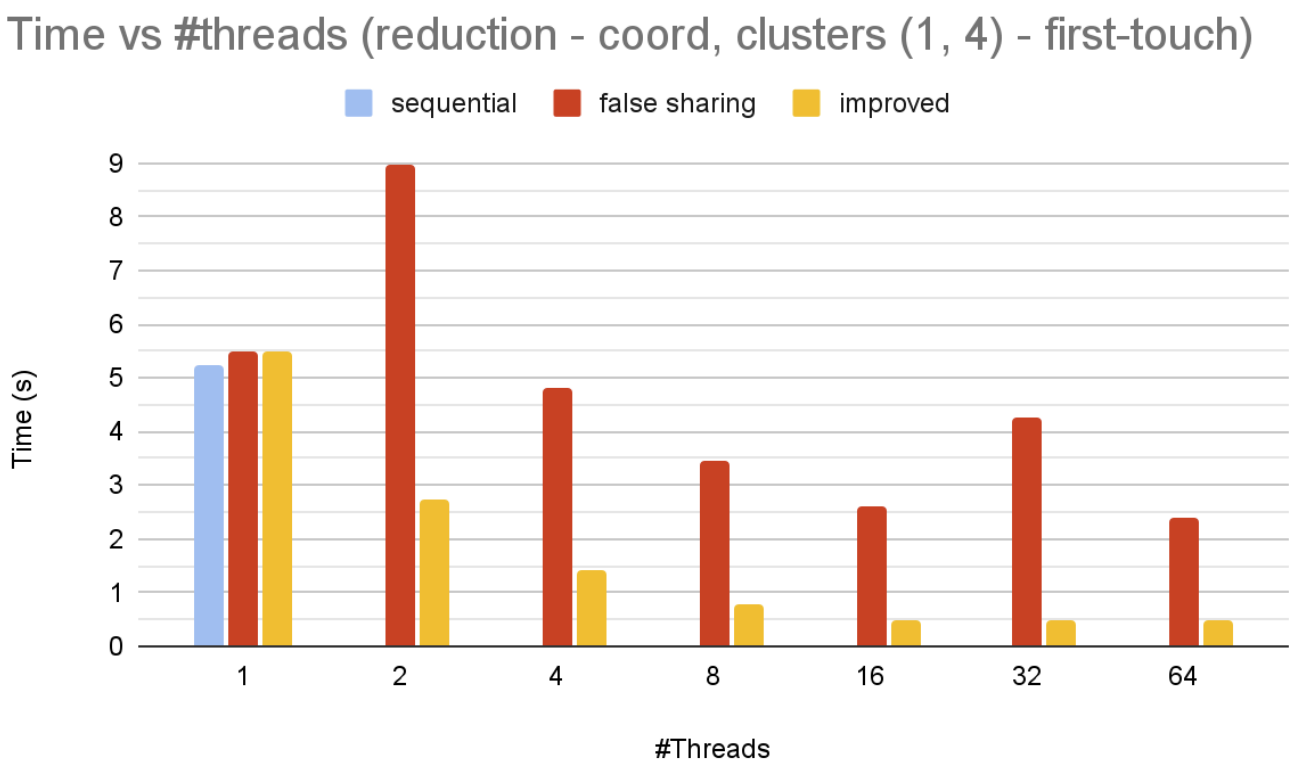
Η πρώτη έννοια με την οποία αξίζει να ασχοληθούμε είναι η first-touch policy του Linux. Πρόκειται για μία στρατηγική κατανομής μνήμης σε συστήματα NUMA, σύμφωνα με την οποία μία σελίδα μνήμης τοποθετείται στο NUMA node του νήματος (ή CPU) που θα την επεξεργαστεί για πρώτη φορά. Έτσι όταν αργότερα ένα άλλο νήμα που ανήκει σε διαφορετικό NUMA node χρειαστεί πρόσβαση σε κάποιο από αυτά τα δεδομένα, θα πρέπει να αποκτήσει πρόσβαση στη μνήμη ενός απομακρυσμένου κόμβου, δυσχεραίνοντας την απόδοση.

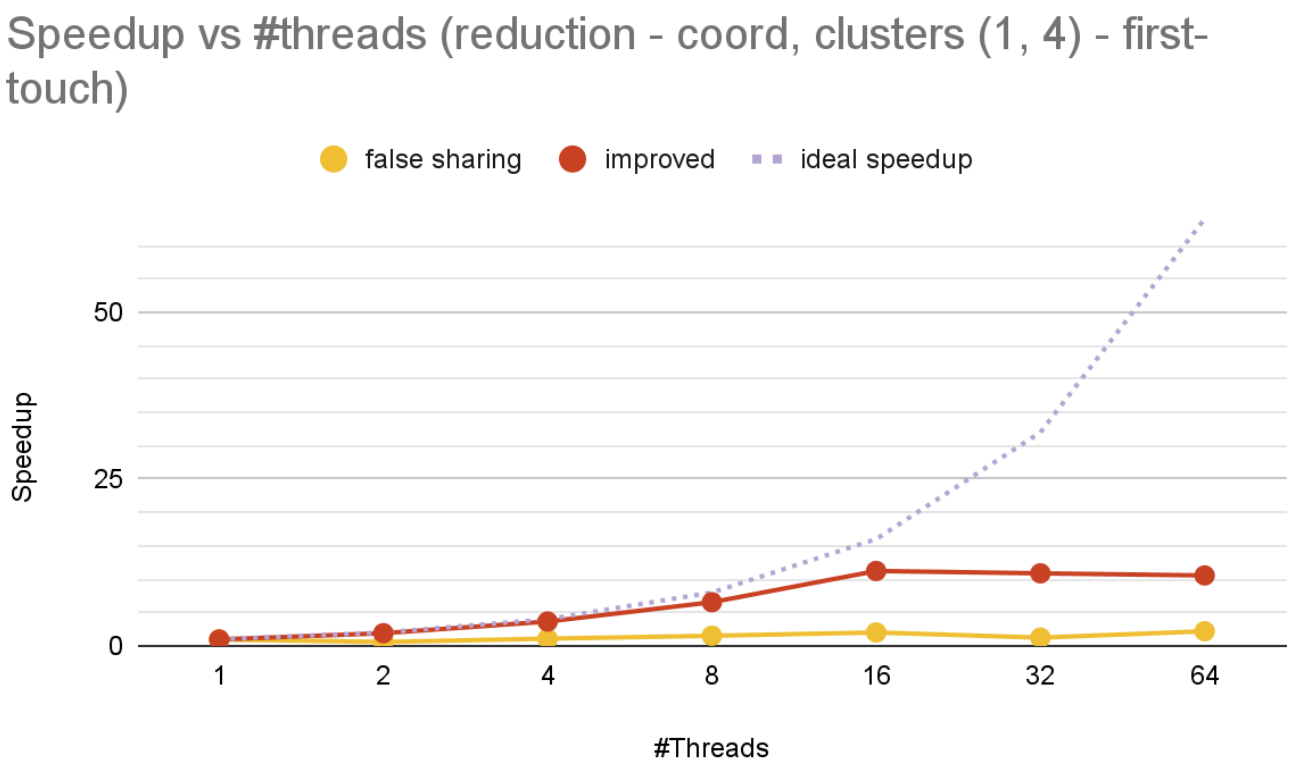
Ένα άλλο φαινόμενο που μας απασχολεί είναι το false-sharing, κατά το οποίο προκαλείται μείωση της απόδοσης όταν πολλά νήματα πραγματοποιούν εγγραφές σε δεδομένα που βρίσκονται στην ίδια cache line. Αυτό έχει ως αποτέλεσμα την επιβολή cache coherence πρωτοκόλλων τα οποία επιφέρουν περιττές καθυστερήσεις.

Είναι λοιπόν αντιληπτό πως προκύπτουν ορισμένα ζητήματα σχετικά με την υπολοποίηση για αυτο το configuration. Αρχικά, στο πρόγραμμα μας όντως οι calloc γίνονται πριν την παράλληλη περιοχή, δηλαδή το master thread (thread 0) είναι υπεύθυνο για την δέσμευση και την αρχικοποίηση των πινάκων local_ClusterSize[ ] και local_Clusters[ ]. Σύμφωνα με την πολιτική first-touch αυτό

έχει ως αποτέλεσμα οι πίνακες αυτοί να βρίσκονται στο NUMA node 0, ανεξάρτητα από το ποιο νήμα είναι υπεύθυνο να τους επεξεργαστεί. Εκεί οφείλεται ένα βασικό κομμάτι της μειωμένης απόδοσης, ενώ ταυτόχρονα εξηγείται και γιατί λάβαμε καλύτερους χρόνους όταν τα νήματα (μέχρι και 16) ήταν τοποθετημένα στον πρώτο κόμβο. Επιπλέον όπως έχουμε πει, οι νέοι πίνακες είναι πολύ μικρότεροι τώρα, με αποτέλεσμα να χωράνε περισσότεροι από ένας σε μία cache line, γεγονός που προκαλεί τα φαινόμενα false-sharing.

Εμείς αξιοποιώντας την στρατηγική first-touch πραγματοποιήσαμε ένα NUMA aware allocation, αντιμετωπίζοντας ταυτόχρονα ένα μέρος του false-sharing που έχει να κάνει με τα κοινά δεδομένα μεταξύ των νημάτων (sharing). Συγκεκριμένα πραγματοποιήσαμε τις malloc και την αρχικοποίηση των local πινάκων σε μία παράλληλη περιοχή, ώστε κάθε νήμα να είναι υπεύθυνο για την δημιουργία του πίνακα τον οποίο θα επεξεργαστεί αργότερα. Τα αποτελέσματα της βελτιωμένης υλοποίησης φαίνονται παρακάτω:





Παρατηρούμε άμεση ελαχιστοποίηση του χρόνου εκτέλεσης καθώς και βελτίωση στην κλιμάκωση μέχρι τα 16 νήματα, όπου το πρόγραμμα φτάνει πιθανώς σε ένα όριο. Τέλος σημειώνεται ότι δοκιμάστηκε άλλη μία τεχνική αντιμετώπισης του false-sharing, το padding στους local πίνακες, ώστε να είναι αρκετά μεγάλοι για να χωράνε σε μία cache line και ναι αποφευχθεί το φαινόμενο. Παρόλα αυτά, ενώ και οι δύο τεχνικές είχαν θετική επίδραση στην επίδοση, ο συνδυασμός τους δεν την βελτίωσε περισσότερο.

2.1.3 Αμοιβαίος Αποκλεισμός - Κλειδώματα

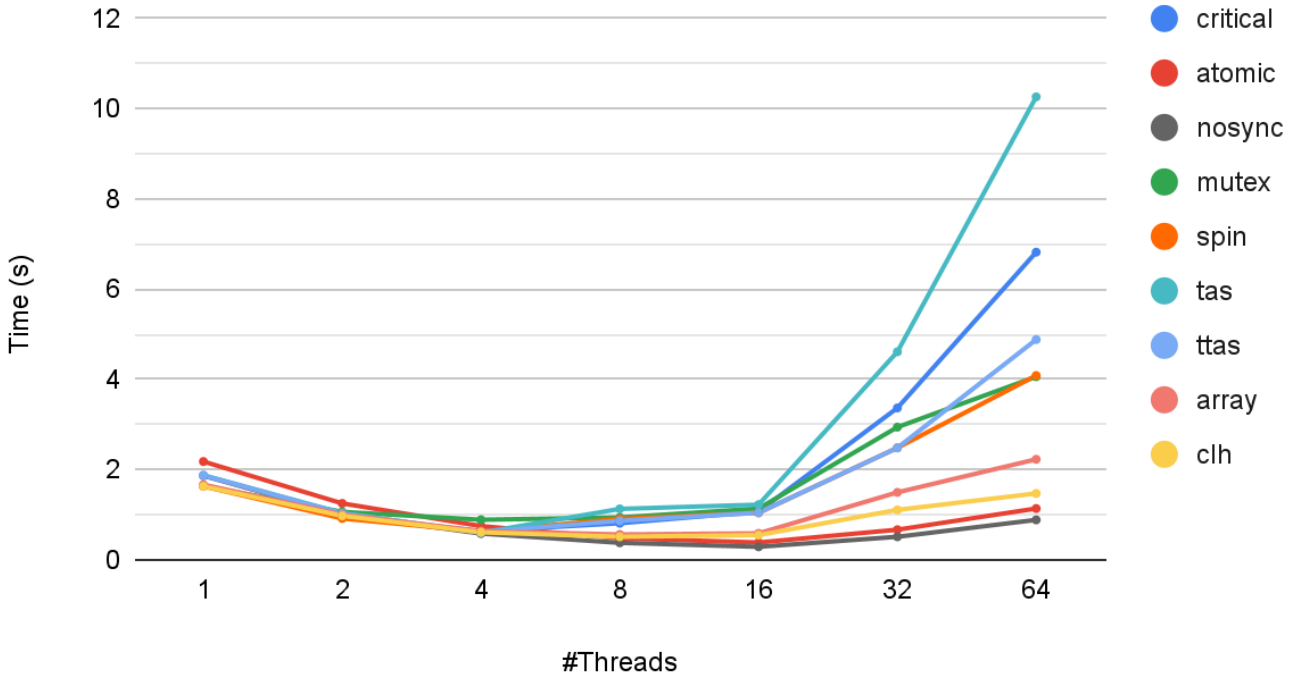
Στην ενότητα αυτή, αξιολογούμε διαφορετικούς τρόπους κλειδώματος για αμοιβαίο αποκλεισμό προστατεύοντας το κρίσιμο τμήμα-ενημέρωση του πίνακα newClusters από την shared clusters υλοποίηση του αλγορίθμου. Συγκεκριμένα μελετήθηκαν τα παρακάτω είδη κλειδωμάτων:

- **nosync_lock:** Η συγκεκριμένη υλοποίηση δεν παρέχει αμοιβαίο αποκλεισμό οπότε δεν παράγει και σωστά αποτελέσματα. Ωστόσο, θα χρησιμοποιηθεί ως άνω όριο για την αξιολόγηση της επίδοσης των υπόλοιπων κλειδωμάτων.
- **pthread_mutex_lock:** πρόκειται για κλείδωμα επιπέδου λειτουργικού συστήματος της βιβλιοθήκης Pthreads. Αν το lock είναι πιασμένο, το thread μπλοκάρει και κοιμάται προκαλώντας context switching.

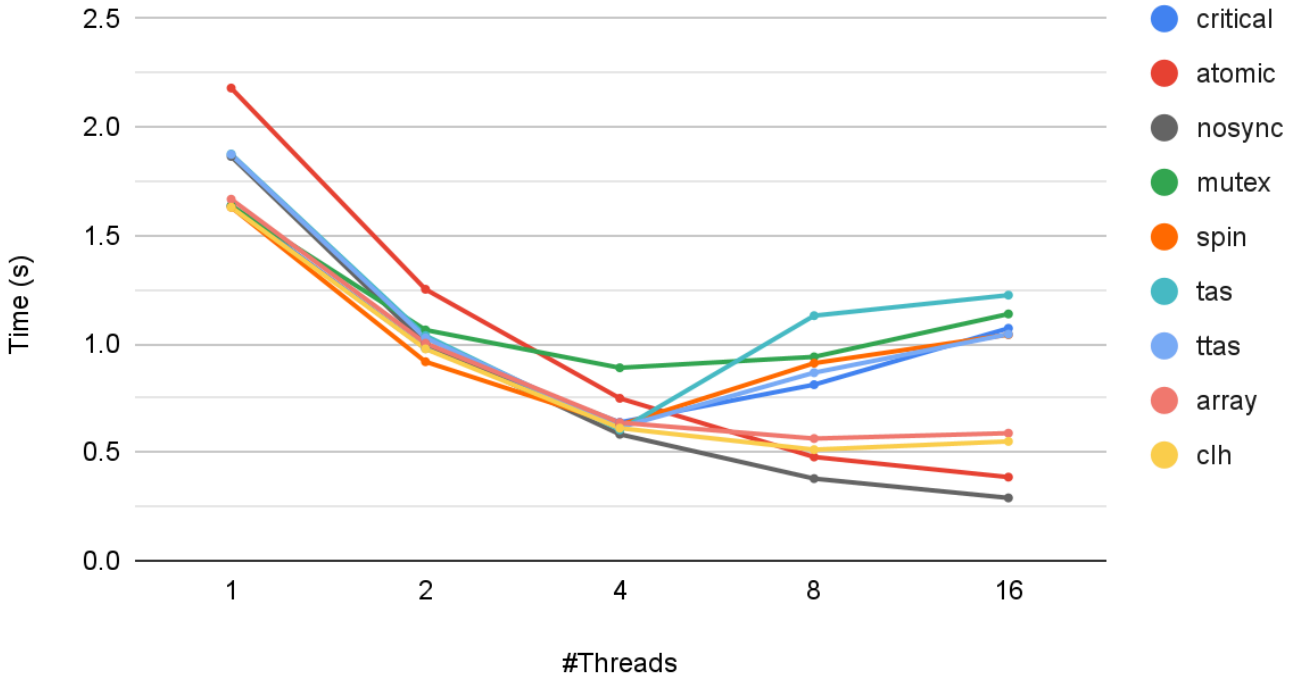
- **pthread_spin_lock:** άλλο ένα κλείδωμα της βιβλιοθήκης Pthreads, με τη διαφορά ότι αντί το νήμα να κοιμηθεί παραμένει ενεργό (busy waiting) ελέγχοντας την τιμή του lock. Έτσι αποφεύγεται το κόστος του context switching.
- **tas_lock (test-and-set):** είναι μία απλή μορφή spinlock που πραγματοποιείται με atomic test-and-set εντολές από κάθε νήμα. Προσπαθώντας να αποκτήσουν το lock, τα νήματα εκτελούν συνεχείς εγγραφές στην ίδια θέση μνήμης προκαλώντας υπερβολική χρήση του bus, λόγω των πρωτοκόλλων cache-coherence.
- **ttas_lock (test-and-test-and-set):** αποτελεί βελτίωση του TAS, καθώς τα νήματα αρχικά διαβάζουν την τιμή του lock χωρίς να γράφουν. Μόνο όταν το lock ελευθερωθεί θα πραγματοποιήσουν test-and-set για να διεκδικήσουν το lock μειώνοντας το coherence traffic.
- **array_lock:** πρόκειται για ένα είδος queue lock στο οποίο κάθε νήμα βρίσκεται και περιμένει στη δική του θέση σε έναν πίνακα-ουρά. Κάθε θέση τοποθετείται σε μία cache line και κάθε νήμα κάνει busy-wait σε δικά του δεδομένα με αποτέλεσμα να αποφεύγεται το false-sharing και ο ανταγωνισμός σε shared δεδομένα, ενώ ταυτόχρονα εξασφαλίζεται ότι όλα τα νήματα θα αποκτήσουν κάποια στιγμή το lock.
- **clh_lock:** είναι άλλο ένα queue-based spinlock στο οποίο κάθε νήμα τοποθετείται σε μία ουρά με μορφή συνδεδεμένης λίστας χρησιμοποιώντας έναν ιδιωτικό κόμβο. Αντίστοιχα, τα νήματα κάνουν busy-wait στον κόμβο του προκατόχου τους χωρίς να χρειάζεται να ενημερώσουν shared δεδομένα.
- **critical/atomic:** επιπλέον χρησιμοποιήθηκαν τα OpenMP directives #pragma omp critical και #pragma omp atomic. Το πρώτο αποτελεί κλείδωμα με mutexes υλοποιημένο από την OpenMP, ενώ το δεύτερο διασφαλίζει ότι το κρίσημο τμήμα θα εκτελεστεί ατομικά.

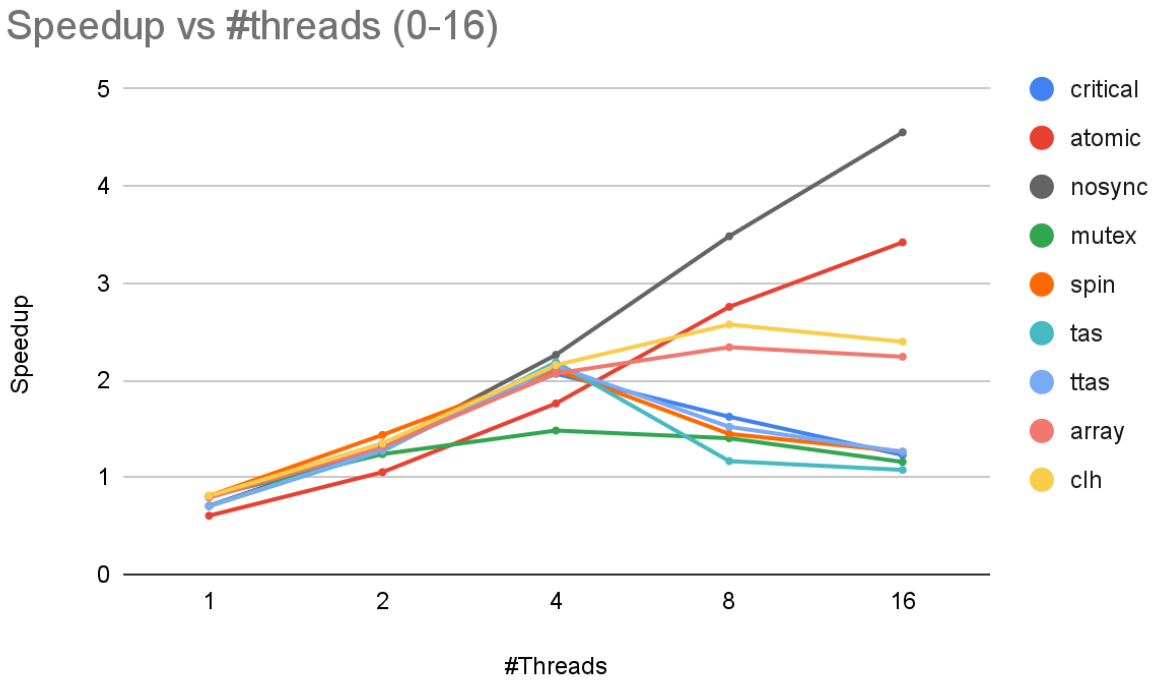
Παρακάτω φαίνονται τα διαγράμματα χρόνου εκτέλεσης του αλγορίθμου με το configuration {Size, Coords, Clusters, Loops} = {32, 16, 32, 10} για threads = {1, 2, 4, 8, 16, 32, 64}. Επειδή για 32 και 64 νήματα εισάγονται περισσότεροι NUMA nodes και η απόδοση χειροτερεύει (για λόγους που εξηγήσαμε στην ενότητα 2.1.1), εστιάζουμε στο κομμάτι 0-16 νήματα:

Time vs #threads



Time vs #threads (0-16)





threads/time	**nosync**	**critical**	**atomic**	**mutex**	**spin**	**tas**	**ttas**	**array**	**clh**
1	1.8647	1.6376	2.18	1.6376	1.6307	1.8757	1.8737	1.6673	1.6305
2	1.0018	0.9785	1.2519	1.0642	0.9177	1.0382	1.0278	1.0029	0.9769
4	0.5831	0.638	0.749	0.8898	0.6257	0.6033	0.6148	0.637	0.6115
8	0.3786	0.8119	0.4784	0.9401	0.911	1.1301	0.8672	0.5634	0.5122
16	0.2896	1.0719	0.3855	1.138	1.0426	1.2248	1.0448	0.5881	0.5502
32	0.5118	3.3676	0.6687	2.9424	2.4839	4.612	2.4834	1.4963	1.1079
64	0.8865	6.8208	1.1385	4.0599	4.0864	10.2628	4.8838	2.2296	1.4727

Αρχικά παρατηρούμε ότι μέχρι τα 4 threads οι ατομικές εντολές είναι κατά κύριο λόγο πιο αργές, ενώ για περισσότερα από 4 νήματα είναι γρηγορότερες από τα locks. Μάλιστα, επειδή η #pragma omp critical εφαρμόζει πλήρη αμοιβαίο αποκλεισμό έχει ως αποτέλεσμα υψηλή καθυστέρηση και κακή κλιμάκωση σε μεγάλο αριθμό νημάτων. Αντίθετα, η #pragma omp atomic συγχρονίζει μόνο μεμονωμένες atomic πράξεις, με σημαντικά μικρότερο overhead.

Όσον αφορά τα κλειδώματα, καλύτερη επίδοση παρουσιάζουν τα queue-based (clh και array) με καλύτερο να αναδεικνύεται το clh, καθώς πετυχαίνουν τους χαμηλότερους χρόνους και διατηρούν ικανοποιητικά το scalability σε σχέση με τα υπόλοιπα. Ακόμα, όπως ήταν αναμενόμενο, το TTAS είναι καλύτερο από το TAS ειδικά όσο αυξάνονται οι πυρήνες, ενώ το δεύτερο παρουσιάζει την χειρότερη κλιμάκωση από όλα λόγω του αυξημένου coherence traffic.

2.2 Παραλληλοποίηση του αλγορίθμου Floyd-Warshall

Σκοπός του δεύτερου μέρους της άσκησης είναι η παραλληλοποίηση του αλγορίθμου Floyd-Warshall, ο οποίος υπολογίζει το ελάχιστο κόστος μεταξύ όλων κόμβων σε ένα γράφο, μέσω του πίνακα γειτνίασης. Ο αλγόριθμος Floyd-Warshall για γράφο με N κόμβους ενεργεί σε πίνακα NxN στοιχείων και εκτελεί N^3 συγκρίσεις min(). Δηλαδή έχει χωρική πολυπλοκότητα $O(N^2)$ και υπολογιστική πολυπλοκότητα $O(N^3)$, πράγμα που καθιστά αναγκαία την παραλληλοποίηση, είτε σε επίπεδο δεδομένων, είτε σε επίπεδο διεργασιών.

2.2.1 Ο αναδρομικός αλγόριθμος Floyd-Warshall

Η πρώτη παράλληλη εκδοχή του αλγορίθμου λειτουργεί αναδρομικά. Αρχικά, χωρίζει τον πίνακα γειτνίασης σε 4 μέρη και αναθέτει τους επιμέρους υπολογισμούς σε threads που συνεχίζουν την αναδρομή, κατά το μέγιστο παράλληλα. Σαν βάση της αναδρομής, χρησιμοποιούμε τον σειριακό αλγόριθμο FW. Αυτό σημαίνει ότι όταν ένας επιμέρους πίνακας αποκτήσει μέγεθος μικρότερο από ένα προκαθορισμένο (μεταβλητή *B*), τότε το αντίστοιχο thread εκτελεί τον "απλό", σειριακό αλγόριθμο. Για την υλοποίηση του αλγόριθμου αξιοποιήθηκαν τα tasks του OpenMP. Τα βασικότερα σημεία του κώδικα φαίνονται παρακάτω:

fw_sr.c:

```
int main(int argc, char **argv)
{
   [...]
   #pragma omp parallel
   {
      #pragma omp single
      {
         FW_SR(A,0,0, A,0,0,A,0,0,N,B);
      }
   }
   [...]
}
[...]



void FW_SR (int **A, int arow, int acol,
            int **B, int brow, int bcol,
            int **C, int crow, int ccol,
            int myN, int bsize)
{
```

```
   int k,i,j;

   if(myN<=bsize)
      for(k=0; k<myN; k++)
         for(i=0; i<myN; i++)
            for(j=0; j<myN; j++)
               A[arow+i][acol+j]=min(A[arow+i][acol+j],

B[brow+i][bcol+k]+C[crow+k][ccol+j]);
   else {
      FW_SR(A,arow, acol,B,brow, bcol,C,crow, ccol, myN/2, bsize);

      #pragma omp task
      FW_SR(A,arow, acol+myN/2,B,brow, bcol,C,crow,
            ccol+myN/2, myN/2, bsize);
      #pragma omp task
      FW_SR(A,arow+myN/2, acol,B,brow+myN/2, bcol,C,crow,
            ccol, myN/2, bsize);

      #pragma omp taskwait

      FW_SR(A,arow+myN/2, acol+myN/2,B,brow+myN/2, bcol,C,crow,
            ccol+myN/2, myN/2, bsize);
      FW_SR(A,arow+myN/2, acol+myN/2,B,brow+myN/2,bcol+myN/2,
            C,crow+myN/2, ccol+myN/2, myN/2, bsize);

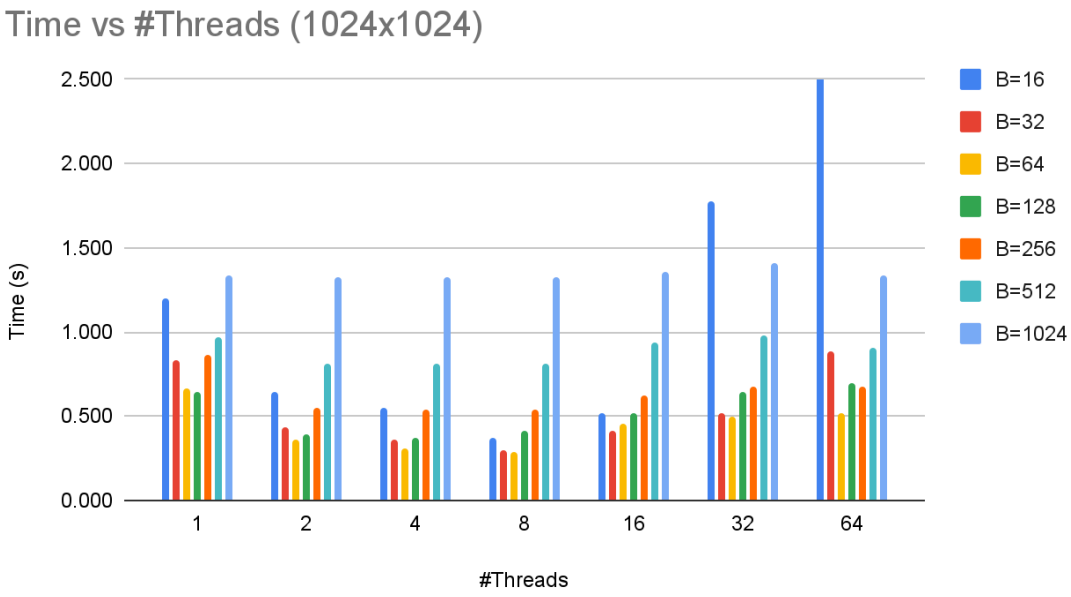
      #pragma omp task
      FW_SR(A,arow+myN/2, acol,B,brow+myN/2, bcol+myN/2,C,
            crow+myN/2, ccol, myN/2, bsize);
      #pragma omp task
      FW_SR(A,arow, acol+myN/2,B,brow, bcol+myN/2,C,
            crow+myN/2, ccol+myN/2, myN/2, bsize);

      #pragma omp taskwait

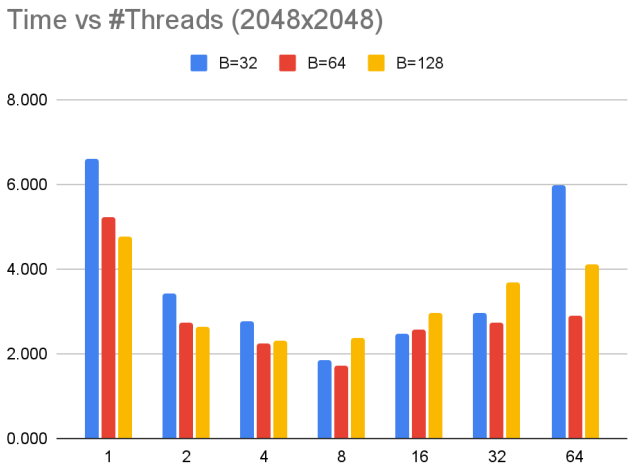
      FW_SR(A,arow, acol,B,brow, bcol+myN/2,C,crow+myN/2,
            ccol, myN/2, bsize);
   }
}
```

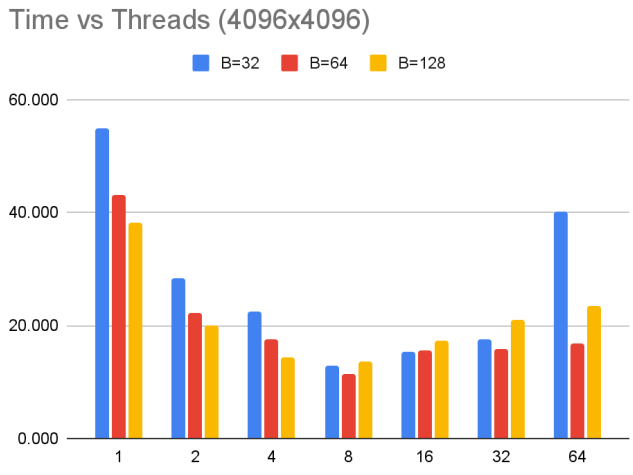
2.2.1.i) Εύρεση βέλτιστου *B*

Αρχικά, αναζητήσαμε το μέγιστο μέγεθος *B*, μετά από το οποίο σταματάει η αναδρομή. Δοκιμάσαμε τιμές *B*=16, 32, 64, 128, 256, 512, 1024 σε σύστημα 1 NUMA node[1], για προβλήματα μεγέθους πίνακα γειτνίασης 1024. Παρακάτω εμφανίζονται τα αποτελέσματα.



Αμέσως παρατηρούμε ότι οι καλύτεροι χρόνοι δίνονται για *B*=32, 64 και 128. Επαναλαμβάνουμε λοιπόν το πείραμα και στα μεγέθη 2048 και 4096, για αυτές τις τιμές του *B*.



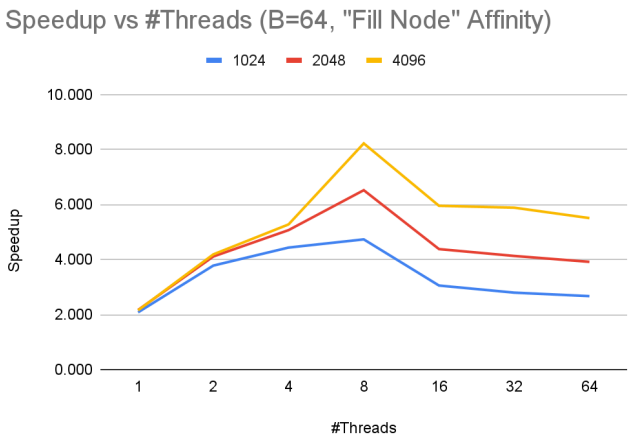


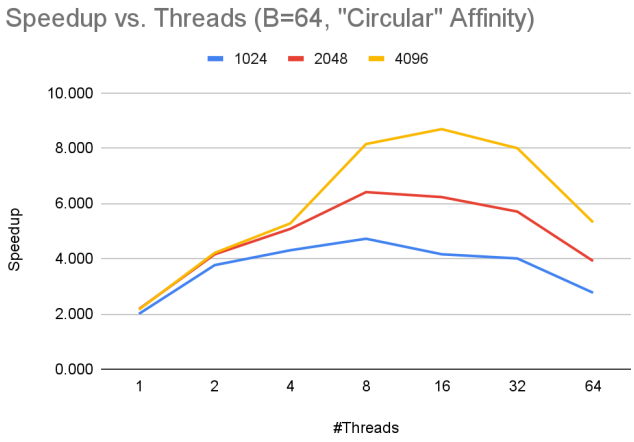
[1] Λέγοντας "1 NUMA node", εννοούμε ότι εκτελούμε το πρόγραμμα σε ένα κόμβο με 1,2,...16 threads και αφού τον "γεμίσουμε", εκτελούμε 32 threads σε δύο κόμβους και 64 threads σε 4 κόμβους. Έχουμε δηλαδή CPU Affinity που "γεμίζει" έναν κόμβο προτού προχωρήσει στον επόμενο.

Από τις μετρήσεις μας μπορούμε να εξάγουμε ενδιαφέροντα συμπεράσματα για την επιρροή του *B* στην επίδοση. Μελετώντας τις ακραίες τιμές, βλέπουμε ότι σε κάθε configuration, ξεκινώντας από *B*=32 έχουμε από τις χειρότερες επιδόσεις, οι οποίες βελτιώνονται καθώς το αυξάνουμε. Έχοντας μικρά *B*, έχουμε περισσότερες αναδρομικές κλήσεις και άρα περισσότερα tasks. Όταν έχουμε υπερβολικά πολλά, το overhead της διαχείρισής τους αρχίζει και επιδρά αρνητικά, κόστος που ενισχύεται καθώς αυξάνουμε τον αριθμό των νημάτων. Από την άλλη μεριά, όταν έχουμε υπερβολικά μεγάλο *B* οι αναδρομικές κλήσεις σταματάνε πολύ νωρίς, συνεχίζοντας με τον σειριακό αλγόριθμο, δυσχαιρένοντας την κλιμάκωση. Ακραία περίπτωση αυτού αποτελεί το *B*=1024 στο πρόβλημα μεγέθους 1024, όπου τρέχει απλά ο σειριακός αλγόριθμος και δεν έχουμε καμία κλιμάκωση. Τελικά, βέλτιστη τιμή του *B* αναδείχθηκε η *B*=64, με την καλύτερη επίδοση να επιτυγχάνεται για όλα τα προβλήματα στα 8 νήματα, με χρόνο 0.292, 1.733 και 11.307 δευτερόλεπτα για πίνακες μεγέθους 1024, 2048 και 4096 αντίστοιχα.

2.2.1.ii) Εύρεση βέλτιστου CPU Affinity configuration

Σε αυτό το στάδιο αναζητούμε το βέλτιστο χρόνο που μπορεί να επιτευχθεί, συγκρίνοντας δύο στρατηγικές κατανομής των νημάτων στο μηχάνημα *Sandman*. Η μία μέθοδος είναι αυτή που χρησιμοποιήθηκε στο προηγούμενο πείραμα, όπου “γεμίζουμε” έναν κόμβο με threads προτού προχωρήσουμε στον επόμενο (στο διάγραμμα, “Fill Node” Affinity). Στην άλλη μέθοδο κατανοούμε τα νήματα κυκλικά, από κόμβο σε κόμβο (“Circular” Affinity)). Στα παρακάτω διαγράμματα συγκρίνονται τα speedup των δύο προσεγγίσεων.





Και στις δύο προσεγγίσεις, μέχρι και τα 8 threads έχουμε το ίδιο, ιδανικό speedup. Αποδίδουμε τη χαμηλή εξάρτηση του προγράμματος από την κατανομή της μνήμης στα μικρά μεγέθη του προβλήματος που δοκιμάζονται. Ακόμα και στη περίπτωση του πίνακα 4096x4096 έχουμε δεδομένα μεγέθους 16 MB, αρκετά μικρά για να χωρέσουν στην cache του επεξεργαστή. Για το λόγο αυτό έχουμε αμελητέες προσβάσεις στη μνήμη και άρα μέχρι τα 8 threads δε βλέπουμε διαφορά.

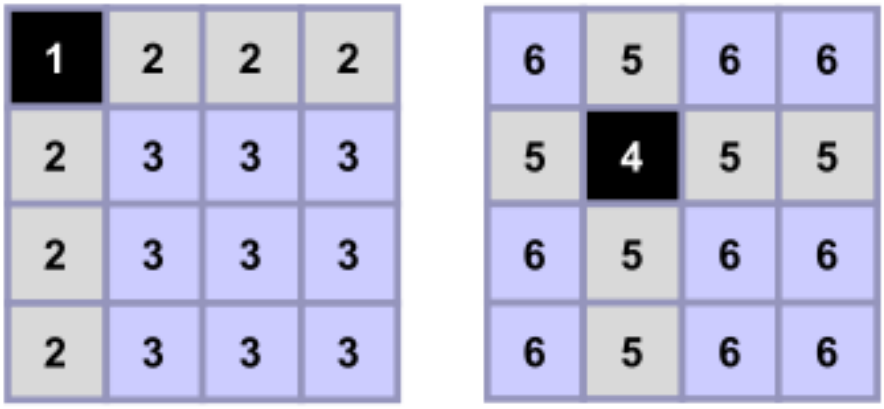
Η διαφορά φαίνεται όταν αυξήσουμε τον αριθμό των threads σε 16, 32 και 64, η οποία οφείλεται στο Hyper-Threading των επεξεργαστών. Στο πρόβλημα που επιλύουμε το Hyper-Threading δε προσφέρει βελτιωμένες ταχύτητες, αφού σε κάθε επεξεργαστή εκτελούνται ταυτόχρονα τα ίδια προγράμματα και επομένως γίνεται χρήση των ίδιων εντολών Assembly, με αποτέλεσμα τις συχνές συγκρούσεις σε Functional Units. Για το λόγο αυτό, το scaling του “Fill Node” Affinity καταρρέει από εκείνο το σημείο και μετά. Στο “Circular” Affinity η αντίστοιχη κλιμάκωση είναι λιγότερο επιδεινωμένη ή ακόμα και βελτιωμένη.

Τελικά, τα καλύτερα speedups συναντώνται για τα προβλήματα μεγέθους 1024 και 2048 στα 8 threads, με τιμές **4.722** και **6.409** αντίστοιχα (οι επιδόσεις ανάμεσα στα affinity schemes είναι όμοιες). Το καλύτερο speedup για το πρόβλημα μεγέθους 4096 επιτυγχάνεται με “Circular” Affinity, στα 16 threads, με τιμή **8.690**.

2.2.2 Ο Tiled αλγόριθμος Floyd-Warshall

Ο τρίτος αλγόριθμος που χρησιμοποιήσαμε είναι ο Tiled Floyd-Warshall. Ο συγκεκριμένος αλγόριθμος χωρίζει τον πίνακα σε χωρία διαστάσεων B και σε κάθε χρονικό βήμα k εκτελεί τρία διακριτά βήματα:

1. Υπολογίζει το μπλοκ [k,k] της διαγωνίου, το λεγόμενο Pivot Block
2. Υπολογίζει τα μπλοκς στην σειρά k και στη στήλη k
3. Υπολογίζει όλα τα υπόλοιπα μπλοκς του πίνακα για το βήμα k



Κάθε βήμα εξαρτάται από το προηγούμενο, όπως επίσης και κάθε χρονικό βήμα k εξαρτάται από το προηγούμενο k-1. Ωστόσο για οποιοδήποτε βήμα από τα 1, 2 ή 3 τα μπλοκς μπορούν να υπολογιστούν εντελώς ανεξάρτητα. Επομένως με χρήση **#pragma omp parallel for** και **#pragma omp barrier** παραλληλοποιήσαμε την εκτέλεσή τους.

fw_tiled.c:

```
[...]
     for(k=0;k<N;k+=B){
          /***** 1. Diagonal square *****/
          FW(A,k,k,k,B);
          /***** 2. Rows and Columns *****/
          #pragma omp parallel for shared(A,k,B)
          for(i=0; i<k; i+=B)
               FW(A,k,i,k,B);

          #pragma omp parallel for shared(A,k,B)
          for(i=k+B; i<N; i+=B)
               FW(A,k,i,k,B);

          #pragma omp parallel for shared(A,k,B)
          for(j=0; j<k; j+=B)
               FW(A,k,k,j,B);

          #pragma omp parallel for shared(A,k,B)
                 for(j=k+B; j<N; j+=B)
                         FW(A,k,k,j,B);
          #pragma omp barrier

          /***** 3. Rest of the table *****/
          for(i=0; i<k; i+=B)
                 #pragma omp parallel for shared(A,k,B)
                 for(j=0; j<k; j+=B)
                      FW(A,k,i,j,B);

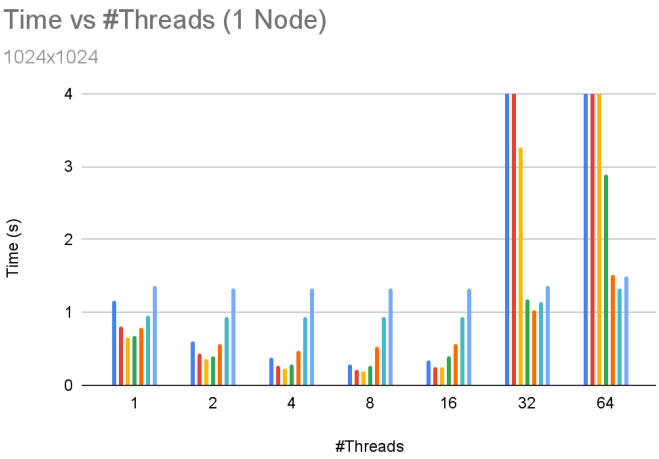
          for(i=0; i<k; i+=B)
                 #pragma omp parallel for shared(A,k,B)
                 for(j=k+B; j<N; j+=B)
                      FW(A,k,i,j,B);

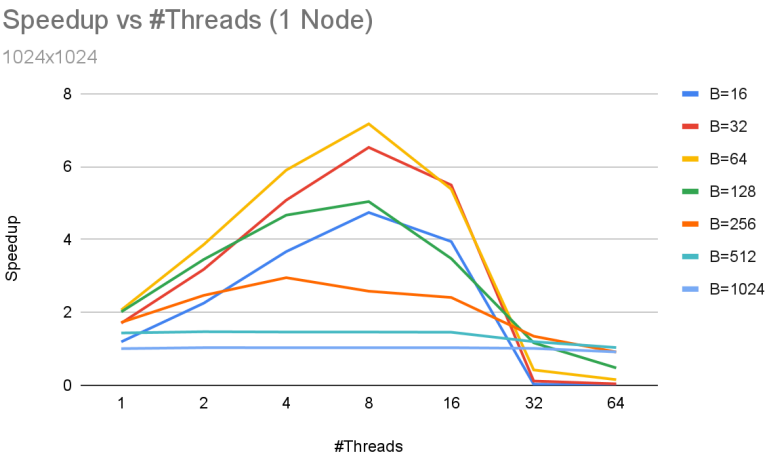
          for(i=k+B; i<N; i+=B)
                 #pragma omp parallel for shared(A,k,B)
                 for(j=0; j<k; j+=B)
                      FW(A,k,i,j,B);

          for(i=k+B; i<N; i+=B)
                 #pragma omp parallel for shared(A,k,B)
                 for(j=k+B; j<N; j+=B)
                      FW(A,k,i,j,B);
          #pragma omp barrier
     }
```

2.2.2.i) Αποτελέσματα σε 1 NUMA node

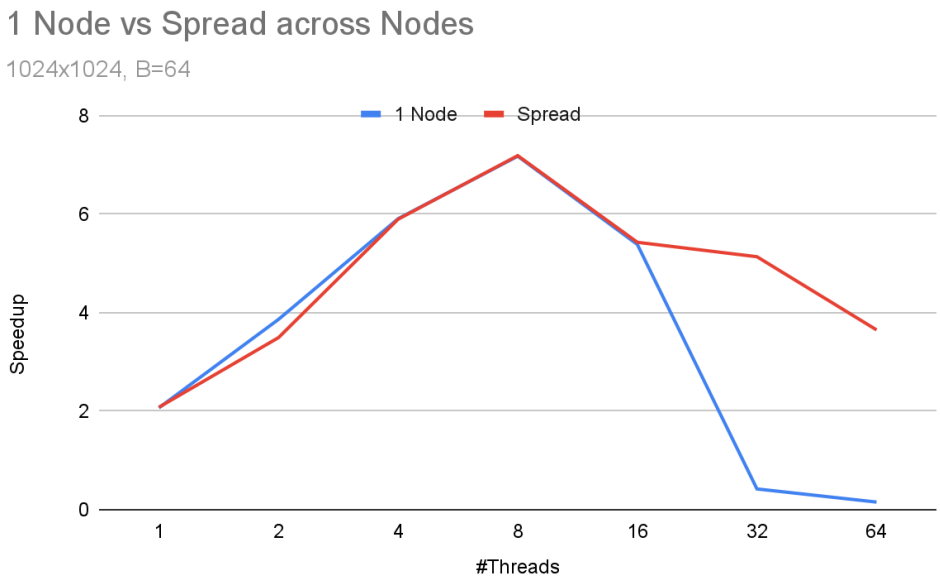
Τρέξαμε τον αλγόριθμο με παραμέτρους τον αριθμό των παράλληλων threads, τις διαστάσεις του πίνακα N και τις διαστάσεις του ελάχιστου μπλοκ B.





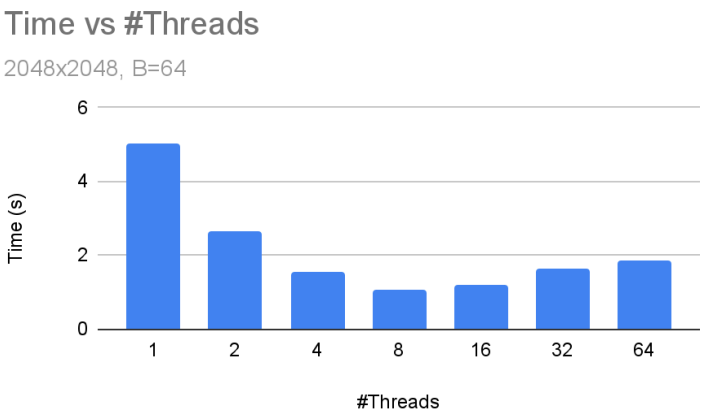
Στα παραπάνω γραφήματα παρουσιάζονται τα αποτελέσματα όταν τρέχουμε όλα τα threads σε **<u>ένα</u>** μόνο NUMA node, με cpu affinity. Παρατηρούμε πως λαμβάνουμε το μέγιστο speedup για 8 threads, δηλαδή για το μέγιστο αριθμό φυσικών πυρήνων του NUMA node. Στα 16 threads, τα επόμενα 8 threads τρέχουν στους 8 virtual πυρήνες. Ωστόσο, επειδή τα physical cores και τα virtual cores μοιράζονται τις L1 και L2 caches και κάθε thread επεξεργάζεται διαφορετικό κομμάτι του πίνακα, η απόδοση μειώνεται λόγω του cache thrashing. Αυξάνοντας τον αριθμό των threads στα 32 και στα 64, η απόδοση μειώνεται ακόμα περισσότερο διότι προστίθεται και το overhead της δρομολόγησης των διεργασιών σε κάθε core.

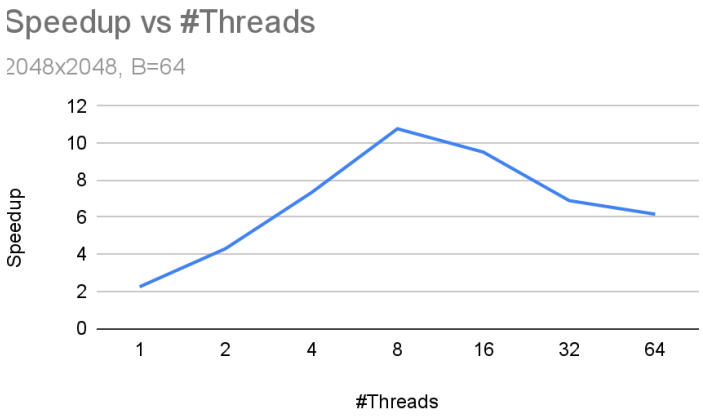
2.2.2.ii) Σύγκριση ενός node με περισσότερα

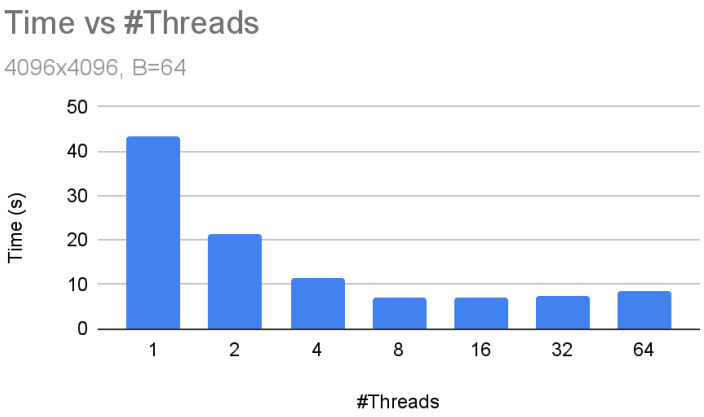


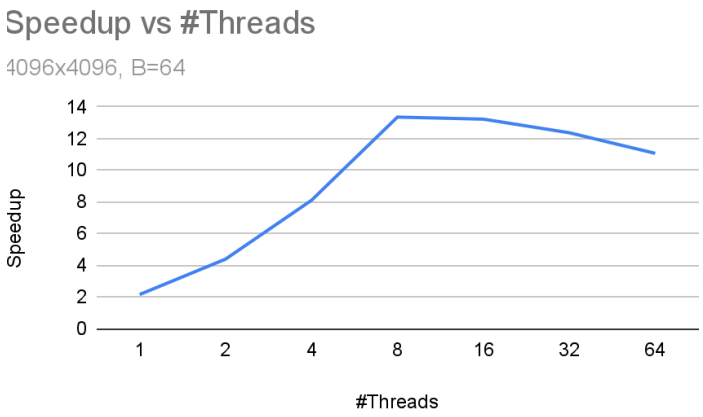
Τρέχουμε πάλι τον αλγόριθμο για N=1024 και B=64, αυτή τη φορά με cpu affinity τέτοιο ώστε τα 16 και 32 threads να μοιραστούν σε 2 πυρήνες και τα 64 σε 4. Το speedup αυξάνεται διότι αξιοποιούμε περισσότερους παράλληλους πυρήνες. Επομένως για τα υπόλοιπα μεγέθη N του πίνακα θα χρησιμοποιήσουμε B=64 και θα μοιράσουμε τα threads στα nodes.

2.2.2.iii) Τελικά συμπεράσματα για τη κλιμάκωση





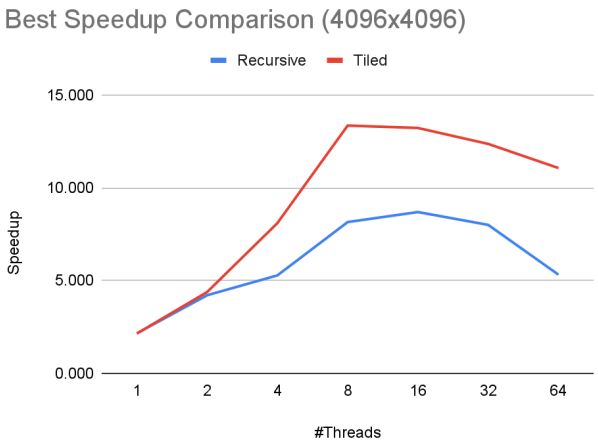




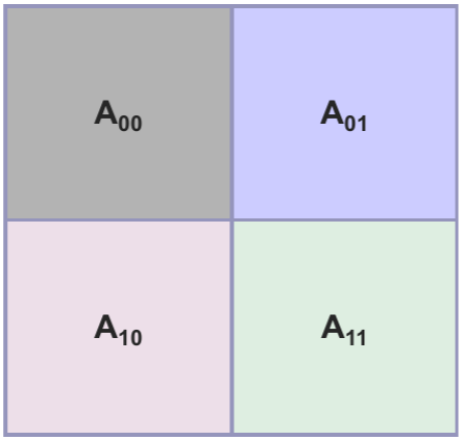
Σύμφωνα με τα παραπάνω διαγράμματα, το πρόγραμμα πράγματι φτάνει το μέγιστο speedup για 8 threads και στη συνέχεια, όταν τα threads μοιράζονται σε διαφορετικά nodes, αρχίζει να μειώνεται. Επομένως μπορούμε να συμπεράνουμε πως το κόστος επικοινωνίας μεταξύ των nodes παίζει καθοριστικό ρόλο στην απόδοση του προγράμματος. Συγκεκριμένα, ένα thread που βρίσκεται στο node 1, θα δουλέψει σε κομμάτια του πίνακα όπου έχουν δουλέψει νωρίτερα threads που βρίσκονται σε άλλα nodes. Η μεταφορά των δεδομένων από απομακρυσμένη μνήμη είναι πολύ χρονοβόρα, ειδικά εάν το interconnect bus έχει υψηλή κίνηση.

2.2.3 Σύγκριση recursive και tiled FW

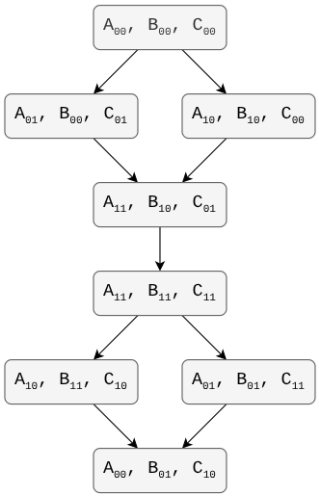
Συνολικά, η εκδοχή του Tiled FW πετυχαίνει πολύ καλύτερα speedups από τον Recursive. Η διαφορά αυτή γίνεται εμφανής στο ακόλουθο διάγραμμα, όπου συγκρίνουμε τις βέλτιστες επιτυχημένες επιδόσεις του κάθε αλγορίθμου στο πρόβλημα μεγέθους 4096x4096.



Αποδίδουμε την αδυναμία του Recursive FW να κλιμακώσει στη χαμηλή παραλληλοποίηση που μπορεί να επιτύχει. Παρακάτω εμφανίζουμε σε ψευδοκώδικα τη διαδικασία δημιουργίας και συγχρονισμού των tasks, μαζί με τον αντίστοιχο task graph των λογικών εξαρτήσεων.




```
FW_SR(A00, B00, C00)
#taskwait
FW_SR(A01, B00, C01)
FW_SR(A10, B10, C00)
#taskwait
FW_SR(A11, B10, C01)
#taskwait
FW_SR(A11, B11, C11)
#taskwait
FW_SR(A10, B11, C10)
FW_SR(A01, B01, C11)
#taskwait
FW_SR(A00, B01, C10)
```



Βλέπουμε ότι σε κάθε χρονική στιγμή εκτελούνται παράλληλα μόνο μέχρι 2 tasks, τα οποία μάλιστα περιμένουν το ένα το άλλο προτού ολοκληρώσουν την εκτέλεσή τους. Αυτό είναι και το χαρακτηριστικό που “καταδικάζει” τον αναδρομικό FW, καθιστώντας τον Tiled τον πιο κατάλληλο για παραλληλοποίηση.

2.3 Ταυτόχρονες δομές δεδομένων

Στο τρίτο πείραμα αυτής της εργαστηριακής άσκησης πραγματοποιείται η σύγκριση ανάμεσα σε δημοφιλείς υλοποιήσεις ταυτόχρονων απλών συνδεδεμένων λιστών. Συγκεκριμένα, εξετάστηκαν οι:

- Coarse-grain locking
- Fine-grain locking
- Optimistic synchronization
- Lazy synchronization
- Non-blocking synchronization

Οι ανωτέρω υλοποιήσεις συγκρίνονται ως προς την επίδοσή τους στα operations `contains()` – *ελέγχουμε αν υπάρχει κάποιο key*, `add()` - *προσθέτουμε key* και `remove()` – *Διαγράφουμε key*. Δοκιμάστηκαν διαφορετικοί συνδυασμοί των operation αυτών, με αναλογίες X-Y-Z: 100-0-0, 80-10-10, 20-40-40 και 0-50-50, όπου X το ποσοστό των `contains()`, Y το ποσοστό των `add()` και Z το ποσοστό των `remove()`.

Βασική μετρική επίδοσης αποτέλεσε το throughput των υλοποιήσεων: το κάθε πρόγραμμα εκτελέστηκε για ακριβώς 10 δευτερόλεπτα, καταμετρώντας το πλήθος των εκτελούμενων operations. Για να έχουμε ουσιαστική εικόνα της επίδοσης, ανεξάρτητα από το μέγεθος του προβλήματος, οι τελικές συγκρίσεις πραγματοποιήθηκαν με βάση το speedup:

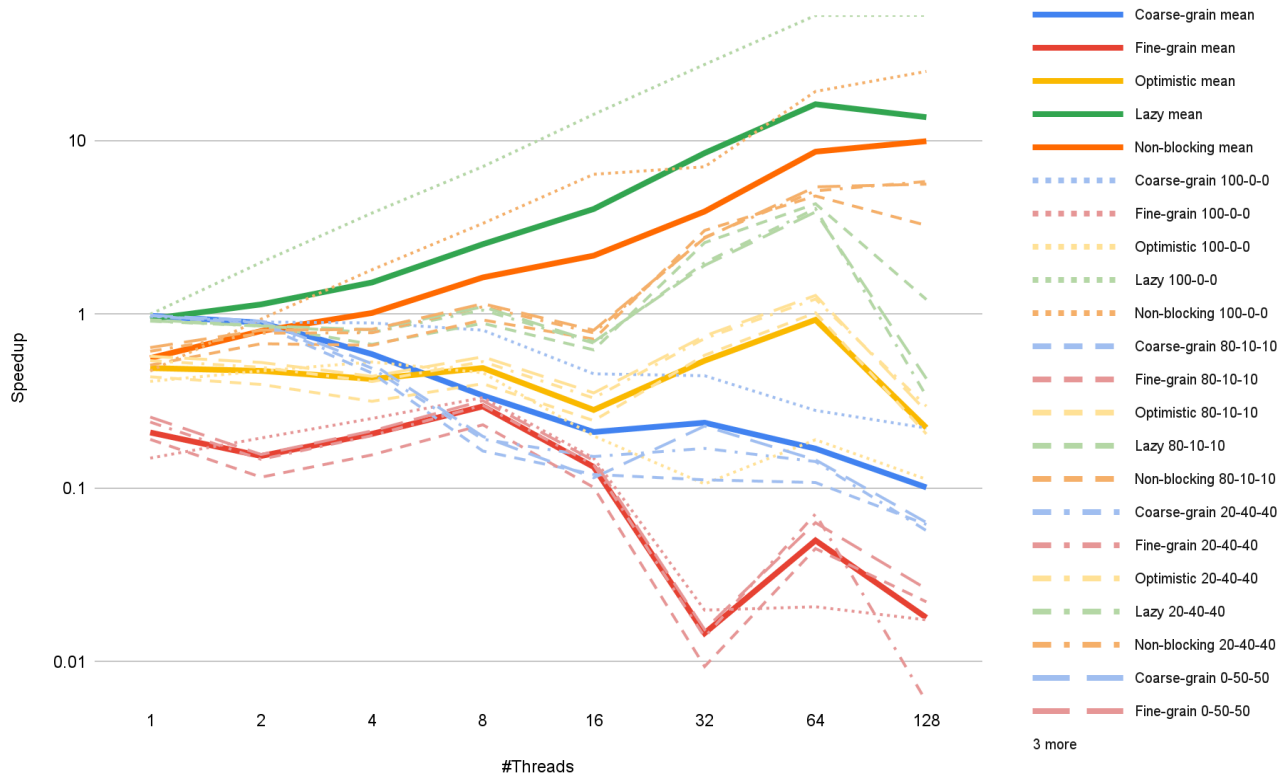
$$speedup = \frac{current\ throughput}{sequential\ throughput}$$

Τα πειράματα εκτελέστηκαν στο μηχάνημα `sandman`, με 4 NUMA nodes, το καθένα εκ των οποίων διαθέτει 8 υπολογιστικούς πυρήνες και δυνατότητα hyperthreading. Για CPU binding χρησιμοποιήθηκε η μεταβλητή περιβάλλοντος `MT_CONF`, αναθέτοντας threads σε διαδοχικούς φυσικούς πυρήνες, «γεμίζοντας» ένα ένα τα NUMA nodes προτού αξιοποιηθεί το hyperthreading.

Παρακάτω φαίνονται τα διαγράμματα της επιτάχυνσης για 1, 2, 4, 8, 16, 32, 64 και 128 νήματα, για μεγέθη λίστας 1024 και 8192 αντίστοιχα:

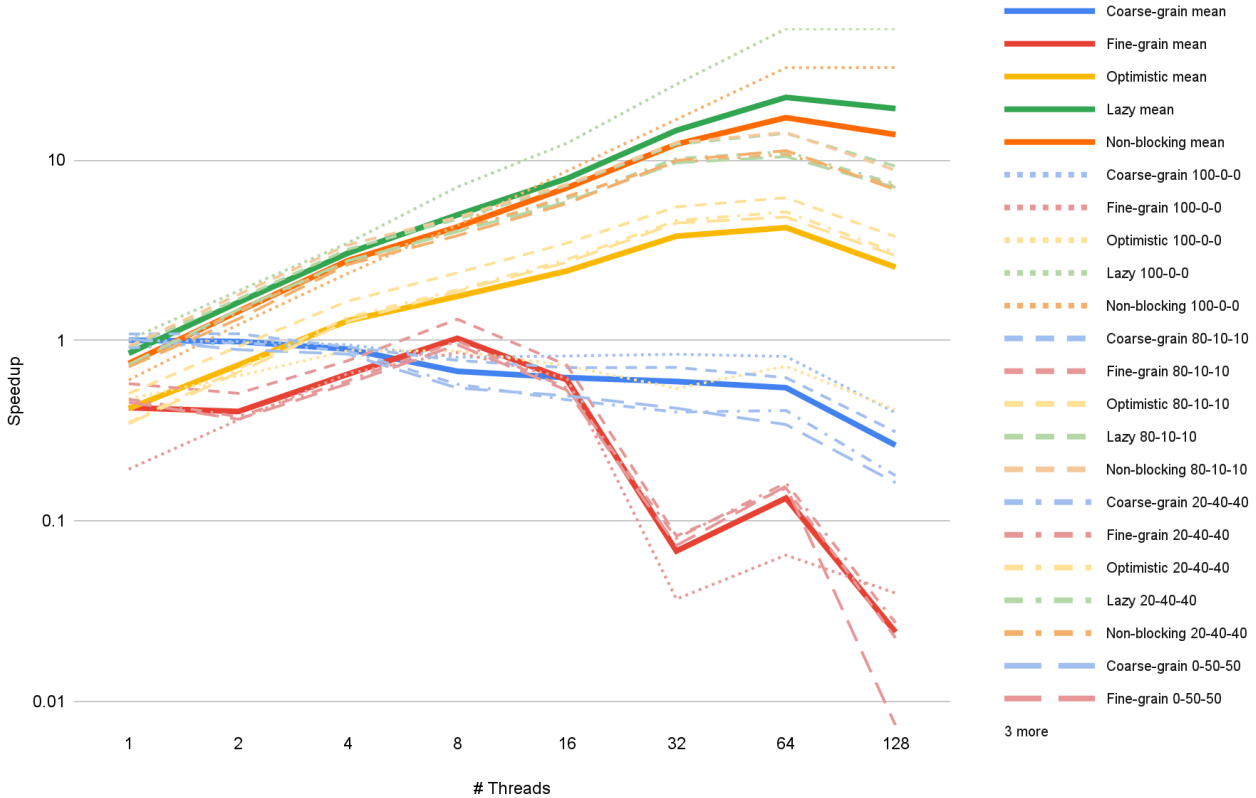
Speedup vs #Threads

List size: 1024



Speedup vs #Threads

List size: 8192



Από τα ανωτέρω διαγράμματα μπορούμε απευθείας να χωρίσουμε, ως προς την αποδοτικότητα, τις προσεγγίσεις σε τρεις ομάδες. Στις λιγότερο αποδοτικές έχουμε τις locking-based, με την coarse-grain locking να επιτυγχάνει καλύτερες επιδόσεις από τη fine-grain locking. Βλέπουμε λοιπόν ότι το να κλειδώσουμε μία φορά ολόκληρη τη λίστα, ακόμα κι αν μηδενίζει την παραλληλοποίησή μας, επιτυγχάνει καλύτερα αποτελέσματα από το να χρησιμοποιούμε συχνά κλειδώματα. Η παρατήρηση αυτή αναδεικνύει το υψηλό κόστος των κλειδωμάτων. Η optimistic synchronization βρίσκεται σε μια ενδιάμεση θέση, με περιορισμένη χρήση locks, αλλά ακριβό validation. Οι καλύτερες επιδόσεις επιτυγχάνονται από τα lazy και non-blocking synchronizations.

Εκ πρώτης όψης, οι καμπύλες μέσων τιμών δείχνουν πως το lazy synchronization είναι αποδοτικότερο από το non-blocking. Εξετάζοντας όμως τις μεμονωμένες καμπύλες, σχεδόν σε όλα τα configurations το non-blocking synchronization πετυχαίνει καλύτερο speedup. Η διαφορά δημιουργείται στο πείραμα που περιλαμβάνει μόνο `contains()` (100-0-0), όπου το lazy synchronization πετυχαίνει με διαφορά τα υψηλότερα throughput. Εξετάζοντας τις υλοποιήσεις των `contains()` των συγκεκριμένων μεθόδων, αποδώσαμε το χάσμα αυτό στους επιπλέον ελέγχους του non-blocking: `is_marked_reference()`, `physical_delete_right()` και `get_unmarked_reference()`. Παρόλο που αυτά αποτελούν κατά κύριο λόγο απλά bit operations, δημιουργούν data dependencies που εμποδίζουν τη βελτιστοποίηση κατά το out-of-order execution, κόστος που αθροίζεται σε μια μεγάλη λίστα.

Όσον αφορά την κλιμάκωση σε σχέση με τον αριθμό των νημάτων, όπως ήταν αναμενόμενο, για το coarse-grain locking παρατηρείται μία σταθερή και μικρή μείωση της επιτάχυνσης καθώς με το κλείδωμα ολόκληρης της λίστας οι πράξεις σειριοποιούνται και προστίθεται επιπλέον overhead. Αντίστοιχα και η fine-grain υλοποίηση εμφανίζει χειρότερο speedup, το οποίο όμως αυξάνεται μέχρι τα 8 threads όπου και γίνεται καλύτερο από την coarse-grain (λογικό αφού πλέον τα operations παραλληλοποιούνται και πολλά νήματα μπορούν να εργάζονται ταυτόχρονα στην λίστα). Στη συνέχεια όμως η επιτάχυνση καταρρέει γεγονός που πιθανόν οφείλεται στην εισαγωγή περισσότερων NUMA nodes και στον έντονο ανταγωνισμό στους πρώτους κόμβους της λίστας.

Επόμενη υλοποίηση είναι η optimistic synchronization η οποία εμφανίζει σαφώς καλύτερη κλιμάκωση από τις προηγούμενες και επίδοση η οποία αναδεικνύεται μετά τα 8 νήματα. Ιδίως στα 16-64 threads παρατηρείται βελτίωση στα update-workloads διότι πλέον τα νήματα μπορουν να προσπελάσουν την λίστα παράλληλα. Αντίθετα όταν πραγματοποιούνται μόνο `contains()`, λόγω των περιττών locks που χρησιμοποιούνται στο validation, η επίδοση υστερεί ακόμα και σε σχέση με την coarse-grain.

Οι lazy και non-blocking synchronization εμφανίζουν την καλύτερη κλιμάκωση αφού χρησιμοποιούν λίγα και καθόλου locks χωρίς να χρειάζεται να διατρέξουν όλη την λίστα για τυχόν validation, αξιοποιώντας έτσι την εισαγωγή περισσότερων νημάτων.

Συνολικά μπορούμε να συμπεράνουμε πως για 1-8 threads επωφελούνται όλες οι παράλληλες υλοποιήσεις και για 16-32 κέρδος έχουν μόνο τα synchronizations καθώς η επικοινωνία μεταξύ NUMA nodes κοστίζει περισσότερο για τα locks. Επιπλέον η αξιοποίηση hyperthreading με 64 νήματα ευνοεί τις παράλληλες υλοποιήσεις, καθώς εκτελούνται προγράμματα με τυχαία operations και άρα συναντώνται λιγότερα FU conflicts. Αντιθέτως, το oversubscription με 128 νήματα σε ελάχιστες περιπτώσεις βελτίωσε την απόδοση.

Τέλος, αυξάνοντας το μέγεθος της λίστας από 1096 σε 8192, βλέπουμε ότι οι επιδόσεις των υλοποιήσεων ως προς το speedup παραμένουν σχετικά όμοιες, με ορισμένες βελτιώσεις. Αυτό είναι σχετικά αναμενόμενο, αφού κατά μέσο όρο θα έχουμε και λιγότερα race conditions. Σημείο προσοχής αποτελεί το non-blocking synchronization, όπου θα αναμέναμε ενδεχομένως χειρότερες επιδόσεις λόγω του αυξημένου κόστους του retry (σε αντίθεση με τα locks, που έχουν προκαθορισμένο κόστος, ανεξάρτητο από το μέγεθος της λίστας). Ο παράγοντας αυτός, όμως, αντισταθμίζεται από τα λιγότερα race conditions και άρα τη σπανιότερη ανάγκη για κλήση retry.

3 Παραλληλοποίηση και βελτιστοποίηση αλγορίθμων σε επεξεργαστές γραφικών

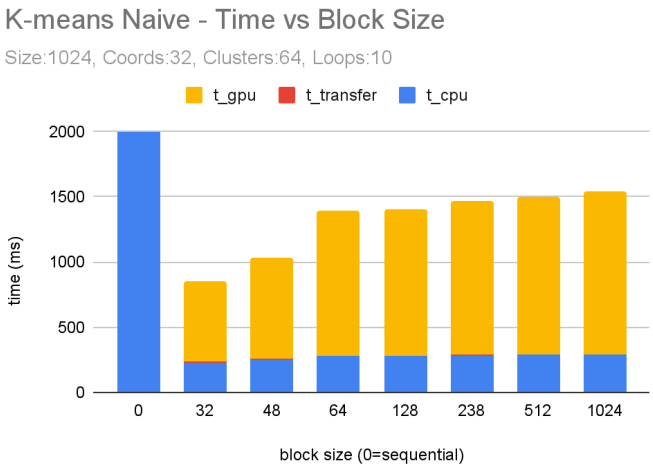
Σκοπός της συγκεκριμένης άσκησης είναι άλλη μία παραλληλοποίηση και βελτιστοποίηση του K-means, αυτή τη φορά σε μια κάρτα γραφικών NVIDIA με χρήση της CUDA.

3.1 K-Means Naive

Ως γνωστόν ο αλγόριθμος αποτελείται από δύο βασικά μέρη, την ανάθεση κάθε αντικειμένου στο κοντινότερο cluster και την ενημέρωση των συντεταγμένων των clusters βάσει των αντικειμένων που ανήκουν σε αυτό. Σε αυτή την υλοποίηση αναθέτουμε στη GPU το πρώτο κομμάτι του αλγορίθμου, που αφορά στον υπολογισμό των nearest clusters για κάθε αντικείμενο.

Σε κάθε επανάληψη, ο πίνακας clusters αντιγράφεται στη GPU, καλείται ο πυρήνας `find_nearest_cluster()` για εκτέλεση στη GPU και τα αποτελέσματα αντιγράφονται πίσω στη CPU. Έπειτα τα clusters ενημερώνονται στη CPU και η διαδικασία επαναλαμβάνεται.

Παρακάτω φαίνεται το διάγραμμα χρόνου εκτέλεσης μιας επανάληψης του αλγορίθμου για διαφορετικά block sizes καθώς και του σειριακού. Να σημειωθεί ότι τόσο σε αυτό, όσο και στα υπόλοιπα μέρη αυτής της άσκησης ο σειριακός αλγόριθμος πέτυχε χρόνο μια τάξη μεγέθους πιο αργό από τις παράλληλες μεθόδους (**6664ms** ανά loop). Για λόγους ευκρίνειας, λοιπόν, επιλέγουμε στα διαγράμματα να κάνουμε περικοπή της μπάρας του σειριακού αλγορίθμου, ώστε να φαίνονται καλύτερα οι διαφορές ανάμεσα στις άλλες μετρήσεις των παράλληλων προγραμμάτων. Τα διαγράμματα speedup όλων των υλοποιήσεών μας συνοψίζονται στο τέλος της ενότητας.



Αρχικά παρατηρούμε σημαντική βελτίωση σε σχέση με την σειριακή έκδοση επιτυγχάνοντας 4-8x επιτάχυνση. Η GPU επιταχύνει το υπολογιστικά βαρύ μέρος του αλγορίθμου, αποδεικνύοντας ότι ο παράλληλος υπολογισμός των αποστάσεων από τα νήματα της GPU συνεισφέρει θετικά στην επίδοση. Όσον αφορά το χρόνο μεταφοράς δεδομένων από τη CPU στη GPU, αυτός αποδείχθηκε αμελητέος.

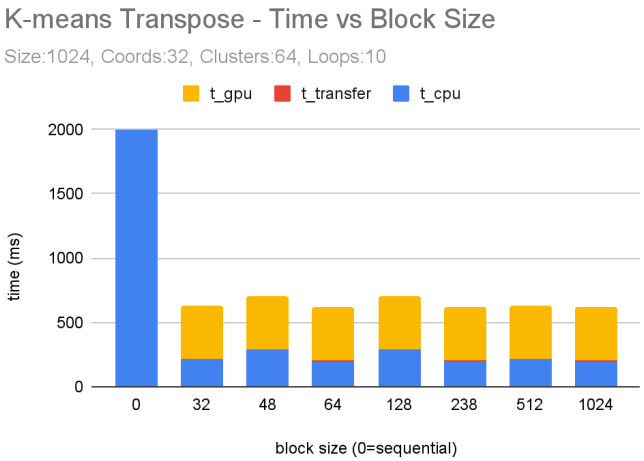
Όσον αφορά το block size, τα καλύτερα αποτελέσματα προκύπτουν για 32, ενώ η αύξησή του χειροτερεύει σταδιακά την επίδοση. Το γεγονός αυτό οφείλεται στη memory-bound φύση του προβλήματος σε συνδυασμό με τον μη αποδοτικό τρόπο πρόσβασης στην μνήμη (uncoalesced access).

Συγκεκριμένα με την χρήση μικρότερων block sizes, περιορίζεται ο αριθμός ενεργών warps (occupancy) που μπορούν να αξιοποιηθούν από το SM καθώς φτάνουμε στο όριο αριθμού blocks που μπορούν να μπουν σε αυτό πριν το όριο αριθμού warps. Αν και συνήθως επιδιόκουμε το μέγιστο δυνατό occupancy, στην συγκεκριμένη περίπτωση αυτό φαίνεται να λειτουργεί αντίστροφα. Έχοντας λιγότερα warps να εκτελούνται ταυτόχρονα, μειώνεται και ο ανταγωνισμός στη μνήμη και η συμφόρηση από αιτήματα μη συνεχόμενων προσπελάσεων.

Γενικά ο kmeans θα μπορούσε να θεωρηθεί ως ιδανικός πυρήνας για GPU καθώς έχει έντονο παραλληλισμό και δεν εμφανίζει divergence, δηλαδή δεν υπάρχουν branches τα οποία θα έθεταν κάποια νήματα του warp ανενεργά. Η συγκεκριμένη όμως υλοποίηση δεν είναι βέλτιστη, καθώς υπάρχουν κι άλλα μέρη του αλγορίθμου που μπορούν να επωφεληθούν από παραλληλοποίηση. Επίσης, μπορούν να πραγματοποιηθούν σημαντικές βελτιστοποιήσεις στον τρόπο που γίνεται πρόσβαση στη μνήμη. Η περίπτωση αυτή θα μελετηθεί στο αμέσως επόμενο μέρος.

3.2 K-Means Transpose

Σε αυτή την έκδοση, η μόνη αλλαγή σε σχέση με τη naive είναι η αλλαγή της δομής των δεδομένων που έχουν διαστάσεις από row-based σε column-based indexing. Τα αποτελέσματα φαίνονται στο παρακάτω διάγραμμα.



Παρατηρούμε εμφανή βελτίωση σε σχέση με την naive έκδοση, με speedup ίσο με 10x. Σχετικά με το block size, η αλλαγή του δεν δείχνει να επηρεάζει την απόδοση επειδή φτάσαμε στο bottleneck του memory bound, δηλαδή την ταχύτητα με την οποία λαμβάνουμε δεδομένα από τη μνήμη.

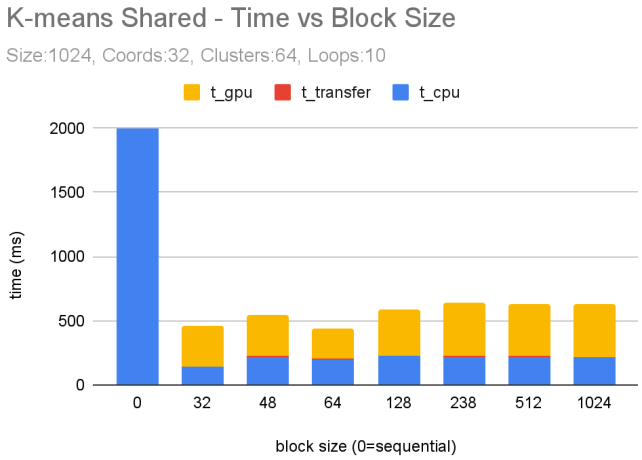
Η σημαντική αυτή διαφορά στην επίδοση μεταξύ των δύο αυτών φαινομενικά αντίστοιχων εκδόσεων οφείλεται στον τρόπο που γίνονται οι προσβάσεις στη μνήμη από τα threads ενός warp που εκτελείται κάθε στιγμή. Κάθε thread επεξεργάζεται ένα αντικείμενο ή ένα cluster. Όταν ένα thread διαβάσει ένα στοιχείο, θα πραγματοποιηθεί ένα memory transaction το οποίο θα φέρει στην cache και τα γειτονικά του στη μνήμη στοιχεία. Στην naive έκδοση, τα στοιχεία αυτά αφορούν το ίδιο αντικείμενο ή cluster και συνεπώς δεν μπορούν να χρησιμοποιηθούν από τα επόμενα threads τα οποία διαβάζουν τα στοιχεία που απέχουν κατά `numCoords` θέσεις στη μνήμη ([numObjs][numCoords], [numClusters][numCoords]). Έτσι για κάθε warp πρέπει να γίνουν περισσότερα memory transactions. Αντίθετα στην transposed έκδοση οι πίνακες είναι της μορφής `[numCoords][numObjs]` και `[numCoords][numClusters]` που σημαίνει πως όταν ένα thread διαβάσει ένα στοιχείο, το block που θα έρθει στη μνήμη περιέχει στοιχεία τα οποία μπορούν να χρησιμοποιηθούν από τα επόμενα threads του warp, δηλαδή γειτονικά threads έχουν πλέον πρόσβαση σε γειτονικές διευθύνσεις μνήμης (coalesced memory accesses).

3.3 K-Means Shared

Στη shared έκδοση του αλγορίθμου τροποποιούμε τον transposed αλγόριθμο και αποθηκεύουμε τα clusters στο shared memory που βρίσκεται σε κάθε Multiprocessor αντί για τη global DRAM του SM. Με αυτόν τον τρόπο έχουμε πολύ πιο γρήγορη πρόσβαση στα δεδομένα των clusters.

Τα δεδομένα που θέλουμε να αποθηκεύσουμε στη shared memory είναι 64 clusters x 32 coordinates x 8 bytes = 16 KB. Από τα hardware specifications γνωρίζουμε πως το Tesla V100

έχει 48-96KB configurable shared memory, άρα γνωρίζουμε πως σε κάθε SM χωράνε το πολύ 3-6 blocks.



Παρατηρούμε ότι έχουμε καταφέρει να ξεπεράσουμε το bottleneck που συναντήσαμε στη transposed υλοποίηση, δηλαδή την αργή RAM. Παρόλο που τα objects βρίσκονται στη global μνήμη, κατά την εκτέλεση ενός thread, απαιτείται μια πρόσβαση σε κάθε coordinate όλων των clusters και κατά συνέπεια σε αυτό το σημείο είχε προκληθεί η μεγαλύτερη καθυστέρηση. Επομένως βλέπουμε ότι για block size = {32, 48, 64} έχουμε ένα speedup ~1.3-2x στον χρόνο εκτέλεσης της gpu σε σχέση με την transposed έκδοση.

Αντίθετα, για block size = {128, 238, 512, 1024} φαίνεται πως έχουμε παραπλήσιες επιδόσεις με την transposed έκδοση. Αυτή η σύγκλιση μας οδηγεί στο τελικό μας συμπέρασμα: και τα δύο προγράμματα καταλήγουν στο bottleneck της πρόσβασης στη global μνήμη για τα δεδομένα των objects.

3.4 Bottleneck Analysis

Προκειμένου να συγκρίνουμε καλύτερα τις υλοποιήσεις μας και να κατανοήσουμε τους μηχανισμούς του προβλήματος, επαναλαμβάνουμε τα προηγούμενα πειράματα, αυτή τη φορά με μόλις 2 διαστάσεις αντί για 32 σε κάθε αντικείμενο. Προκειμένου να διατηρηθεί το μέγεθος του dataset σταθερό, το πρόγραμμά μας θα δημιουργήσει περισσότερα objects, σύμφωνα με τον ακόλουθο τύπο:

$$numObjs = \frac{dataset_size \times 1024 \times 1024}{numCoords \times sizeof(double)}$$

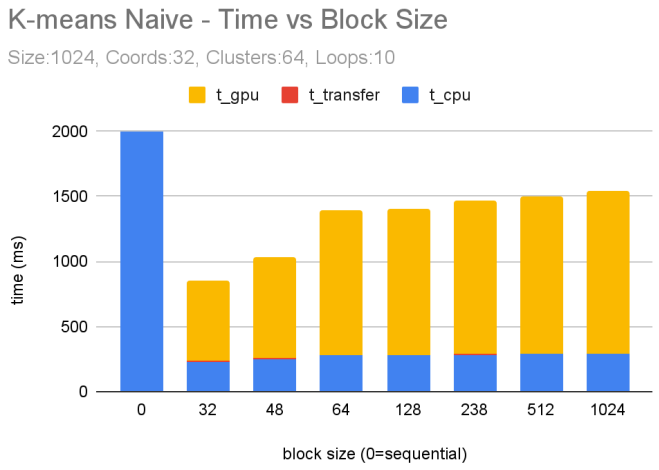
από όπου προκύπτει ότι έχουμε συνολικά x16 περισσότερα σημεία.

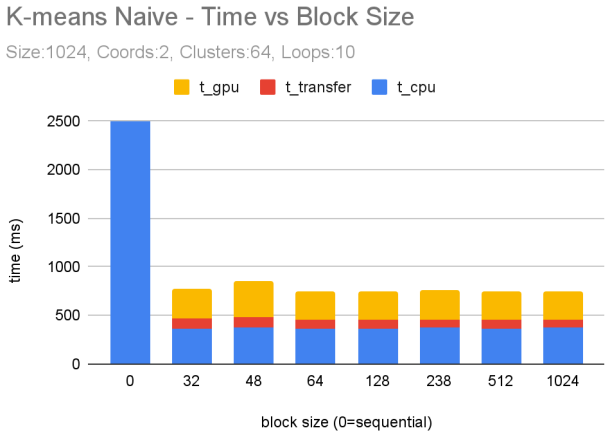
Στα παρακάτω διαγράμματα εφμανίζουμε τα αποτελέσματα του νέου μας πειράματος. Στην αριστερή στήλη εμφανίζουμε ξανά τα αποτελέσματα των πρώτων πειραμάτων μας, για λόγους ευκολότερης σύγκρισης.

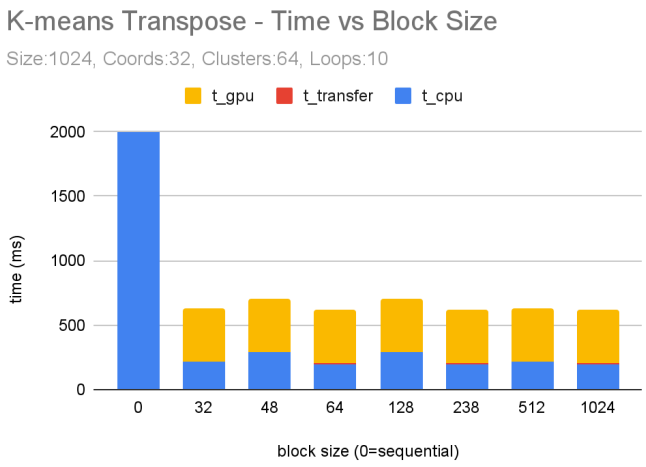
Εστιάζοντας για λίγο στα αρχικά μας πειράματα, μπορούμε να εξάγουμε μερικά πρώτα συμπεράσματα σχετικά με την επιρροή διαφόρων παραγόντων στην επίδοση του επαναληπτικού μέρους του αλγορίθμου. Αρχικά, ο χρόνος μεταφοράς δεδομένων από τη CPU στη GPU είναι σε όλες τις περιπτώσεις αμελητέος. Οι βελτιώσεις του transpose (πρόσβαση σε γειτονικές διευθύνσεις μνήμης) έκανε την μεγαλύτερη διαφορά στη βελτίωση του χρόνου, ενώ οι βελτιστοποιήσεις της κοινής πρόσβασης στα clusters επιφέρει λίγη ακόμα βελτίωση. Προφανώς, οι βελτιώσεις αυτές μειώνουν τον χρόνο εκτέλεσης σε GPU, ενώ οι χρόνοι εκτέλεσης σε CPU παραμένουν σταθεροί.

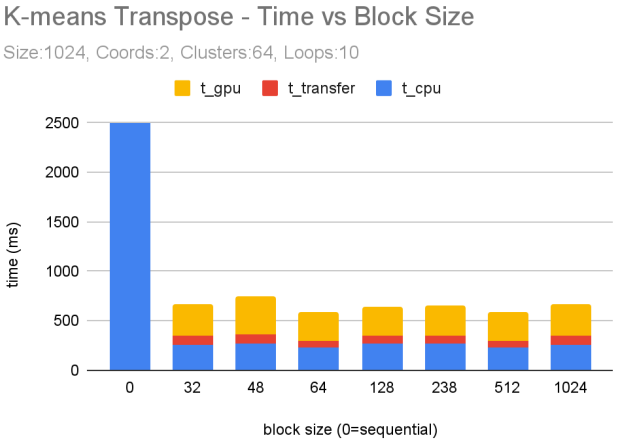
Θεωρούμε ότι οι σταθεροί αυτοί χρόνοι της CPU είναι και το bottleneck του προγράμματος. Για να πετύχουμε καλύτερους χρόνους θα πρέπει είτε με κάποιο τρόπο να πραγματοποιηθεί η ανανέωση των κεντροειδών ταχύτερα στον επεξεργαστή, είτε να ανατεθεί και αυτό στη GPU.

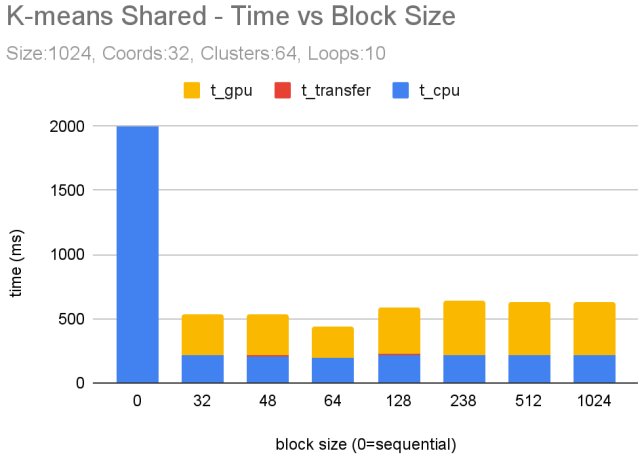
Εξετάζοντας τώρα τα αποτελέσματα του νέου μας πειράματος, εντοπίζουμε ορισμένες διαφορές στους χρόνους εκτέλεσης, που αιτιολογούνται πλήρως από τα χαρακτηριστικά του νέου προβλήματος, ιδίως το γεγονός ότι πλέον έχουμε περισσότερα σημεία, δύο μόνο διαστάσεων. Αρχικά, ο σειριακός χρόνος είναι σημαντικά μεγαλύτερος, πάνω από διπλάσιος (από **6664 ms** ανά loop, ανέβηκε στα **15369 ms** ανά loop).

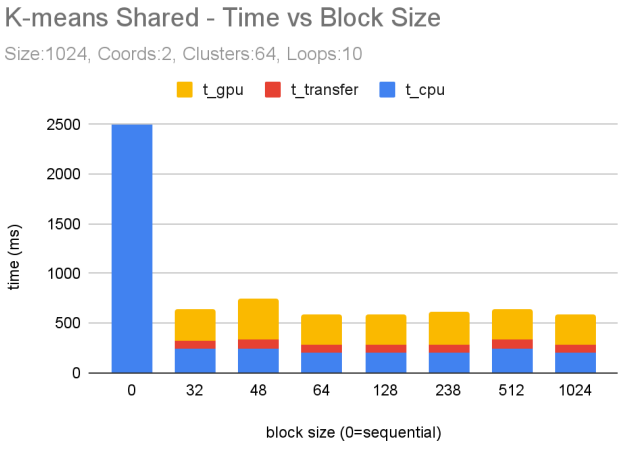

K-means Naive - Time vs Block Size
Size:1024, Coords:32, Clusters:64, Loops:10
t_gpu
t_transfer
t_cpu
2000
1500
1000
500
0
time (ms)
0
32
48
64
128
238
512
1024
block size (0=sequential)


K-means Naive - Time vs Block Size
Size:1024, Coords:2, Clusters:64, Loops:10
t_gpu
t_transfer
t_cpu
2500
2000
1500
1000
500
0
time (ms)
0
32
48
64
128
238
512
1024
block size (0=sequential)


K-means Transpose - Time vs Block Size
Size:1024, Coords:32, Clusters:64, Loops:10
t_gpu
t_transfer
t_cpu
2000
1500
1000
500
0
time (ms)
0
32
48
64
128
238
512
1024
block size (0=sequential)


K-means Transpose - Time vs Block Size
Size:1024, Coords:2, Clusters:64, Loops:10
t_gpu
t_transfer
t_cpu
2500
2000
1500
1000
500
0
time (ms)
0
32
48
64
128
238
512
1024
block size (0=sequential)


K-means Shared - Time vs Block Size
Size:1024, Coords:32, Clusters:64, Loops:10
t_gpu
t_transfer
t_cpu
2000
1500
1000
500
0
time (ms)
0
32
48
64
128
238
512
1024
block size (0=sequential)


K-means Shared - Time vs Block Size
Size:1024, Coords:2, Clusters:64, Loops:10
t_gpu
t_transfer
t_cpu
2500
2000
1500
1000
500
0
time (ms)
0
32
48
64
128
238
512
1024
block size (0=sequential)

Προχωρώντας τους παράλληλους αλγορίθμους, αμέσως βλέπουμε δραστική βελτίωση. Βασικό αίτιο είναι ότι αυξάνεται το παράλληλο τμήμα του κώδικα. Το `find_nearest_cluster()` kernel λειτουργεί παράλληλα ως προς τον αριθμό των αντικειμένων, αλλά σειριακά ως προς τον αριθμό διαστάσεων. Με το νέο configuration λοιπόν επιτρέπουμε στην GPU να φτιάξει πολύ περισσότερα threads, όπου το καθένα εκτελείται σημαντικά πιο γρήγορα.

Μία ακόμα παρατήρηση είναι ότι ο χρόνος μεταφοράς δεδομένων αυξάνεται αρκετά και δεν είναι πλέον αμελητέος. Αποδίδουμε την περαιτέρω αυτή καθυστέρηση στο μεγαλύτερο πλήθος σημείων και συγκεκριμένα στον πίνακα `deviceMembership`, οποίος αντιγράφεται από τη μνήμη της GPU στη μνήμη της CPU μετά τη λήξη του `find_nearest_cluster()`.

3.5 K-Means All-GPU

Σε αυτό το στάδιο μεγιστοποιούμε το ποσοστό των εργασιών που αναθέτουμε στη GPU. Διατηρώντας τη χρήση γειτονικών θέσεων μνήμης και την αποθήκευση των clusters στην κοινή μνήμη, προσθέτουμε το kernel `update_centroids()`, που υπολογίζει παράλληλα τα νέα κέντρα των clusters.

```
void find_nearest_cluster(int numCoords,
                          int numObjs,
                          int numClusters,
                          double *deviceobjects,
                          double *devicenewClusters,
                          int *devicenewClusterSize,
                          double *deviceClusters,
                          int *deviceMembership,
                          double *devdelta) {
  extern __shared__ double shmemClusters[];

  for(int i = threadIdx.x; i < numCoords*numClusters; i += blockDim.x)
      shmemClusters[i] = deviceClusters[i];
  __syncthreads();

  int tid = get_tid();

  if (tid < numObjs) {
    int index, i;
    double dist, min_dist;

    // find nearest cluster
    index = 0;
    min_dist = euclid_dist_2_transpose(numCoords, numObjs, numClusters,
                                       deviceobjects, shmemClusters, tid,
0);
    for (i = 1; i < numClusters; i++) {
```

```
      dist = euclid_dist_2_transpose(numCoords, numObjs, numClusters,
                                     deviceobjects, shmemClusters, tid,
i);

      if (dist < min_dist) {
        min_dist = dist;
        index = i;
      }
    }

    // update cluster membership
    if (deviceMembership[tid] != index) {
      atomicAdd(devdelta, 1.0);
    }

    deviceMembership[tid] = index;

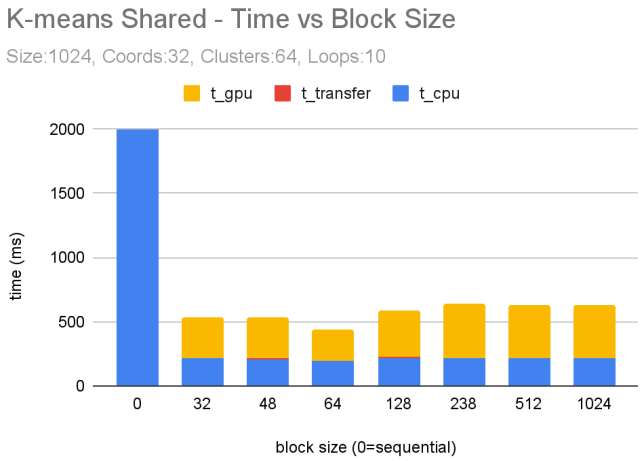
    atomicAdd(&devicenewClusterSize[index], 1);

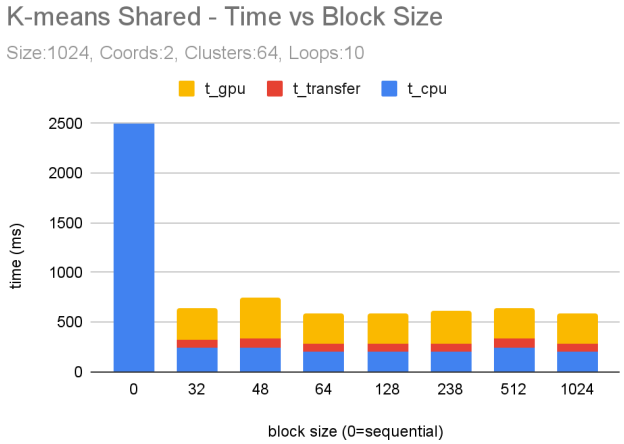
    // add object to cluster's  center
    for(int i = 0; i < numCoords; i++) {
        atomicAdd(&devicenewClusters[i * numClusters + index],
                  deviceobjects[i * numObjs + tid]);
    }
  }
}
```

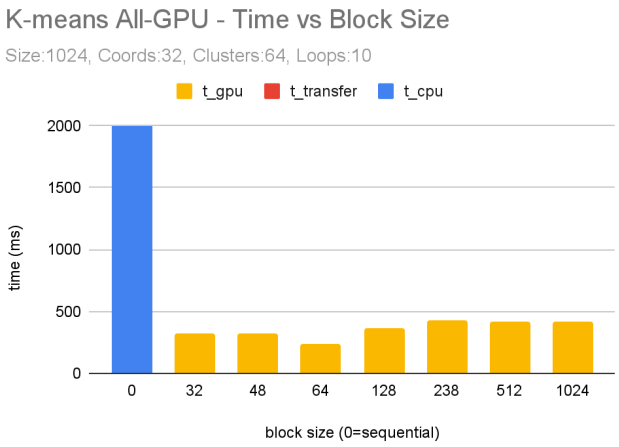
```
void update_centroids(int numCoords,
                      int numClusters,
                      int *devicenewClusterSize,
                      double *devicenewClusters,
                      double *deviceClusters)
{
  int tid = blockIdx.x * blockDim.x + threadIdx.x;
  int totElements = numCoords * numClusters;

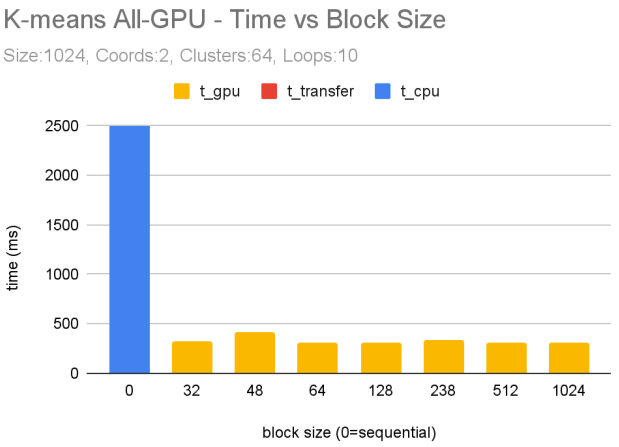
  if (tid < totElements){
    int clusterId = tid % numClusters;
    int count = devicenewClusterSize[clusterId];
    if (count > 0) {
      // the cluster centers are currently the sum of their points
      // we just need to divide them by the number of points
      deviceClusters[tid] = devicenewClusters[tid] / count;
    }
    devicenewClusters[tid] = 0;
  }
}
```

Στο παραπάνω απόσπασμα κώδικα, φαίνονται τα δύο kernels που υλοποιούν τη διεξαγωγή του K-Means στη GPU: `find_nearest_clsuter()` και `update_centroids()`. Στο kernel `find_nearest_clsuter()`, κάθε thread αναλαμβάνει από ένα αντικείμενο, υπολογίζει το κοντινότερό του cluster και αναλαμβάνει την αλλαγή cluster, σε περίπτωση που προκύψει. Σε αντίθεση με τις προηγούμενες υλοποιήσεις, στο kernel αυτό αναθέτουμε και μέρος της ανανέωσης των κεντροειδών των clusters. Συγκεκριμένα, μετά την επιλογή του νέου cluster ενός αντικειμένου, το kernel προσθέτει απευθείας τις συντεταγμένες του στο κέντρο του νέου cluster. Έτσι, το `update_centroids()` kernel αντί να υπολογίσει τον μέσο όρο από την αρχή, το μόνο που έχει να κάνει είναι η διαίρεση με το πλήθος των μελών. Με αυτήν τη μέθοδο, μπορούμε να αναθέσουμε σε κάθε thread του `update_centroids()` από ένα cluster και αντί να πραγματοποιήσουμε τα αθροίσματα σε for loop, να τα έχουμε ήδη εκτελέσει παράλληλα σε object-level (κατά την `find_nearest_clsuter()`).





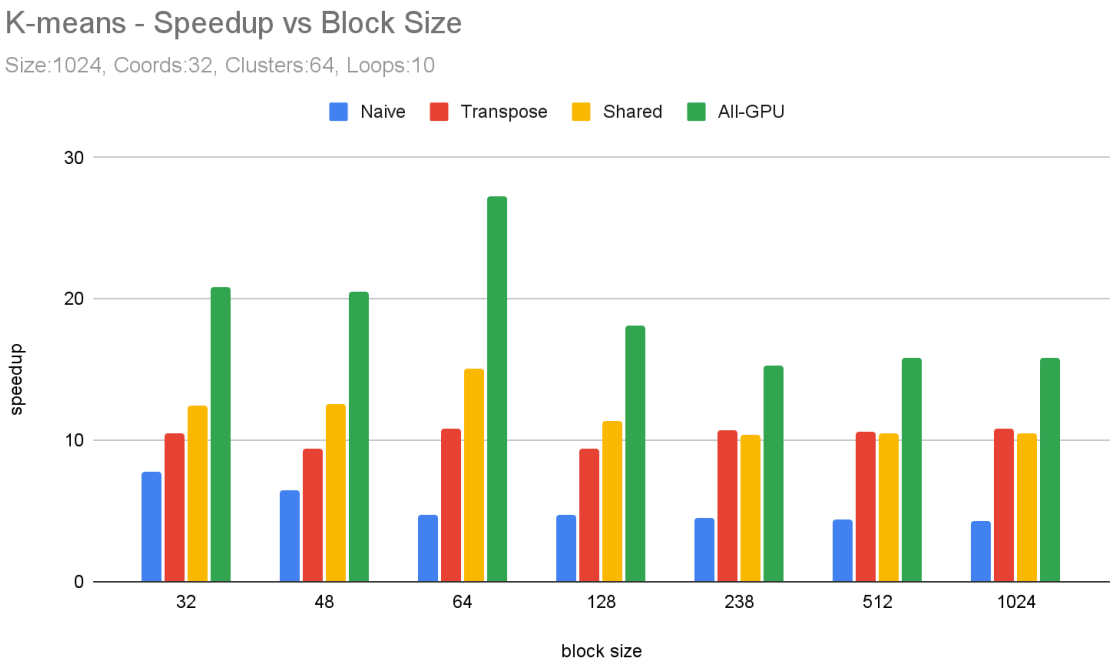


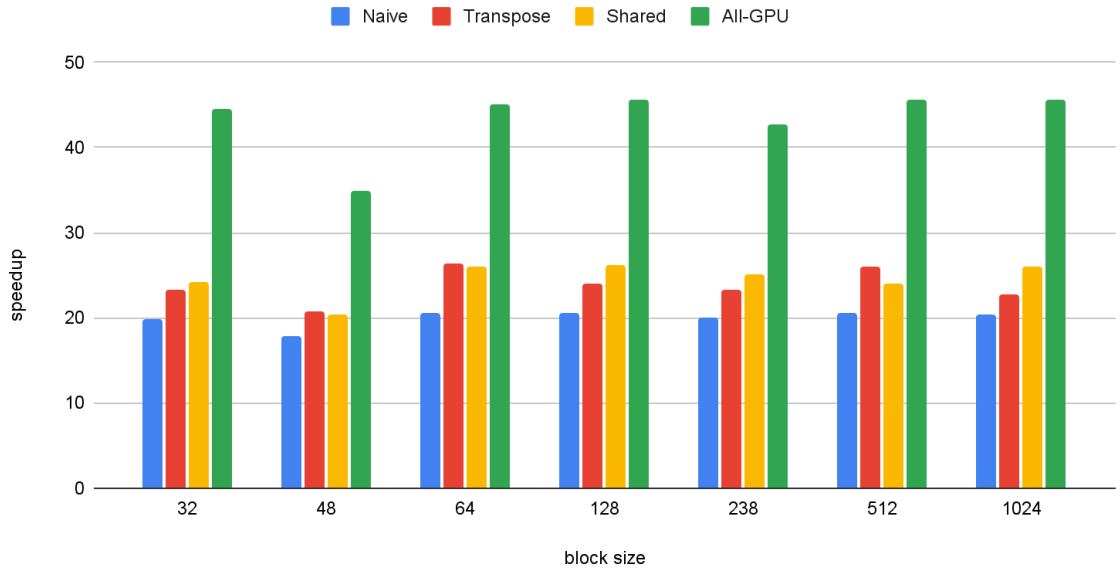


Στα παραπάνω διαγράμματα φαίνονται οι μέσοι χρόνοι εκτέλεσης ενός loop και για τα δύο configurations. Για λόγους σύγκρισης, εμφανίζουμε ξανά τα αποτελέσματα του K-Means Shared. Αμέσως βλέπουμε ότι με τη νέα μέθοδο ξεπερνάμε το bottleneck: μηδενίζουμε το χρόνο εκτέλεσης σε CPU και μεταφοράς δεδομένων. Ταυτόχρονα, οι επιπλέον λειτουργίες που αναθέσαμε στην GPU φαίνεται να μην επηρεάζουν το χρόνο εκτέλεσής της, πετυχαίνοντας συνολικά περίπου το μισό χρόνο σε σχέση με τις προηγούμενες προσεγγίσεις.

Η επιτυχία αυτής της προσέγγισης οφείλεται στην `update_centroids()` και την καταλληλότητα αυτής για εκτέλεση σε GPU. Ο υπολογισμός των νέων κεντροειδών είναι τύπου SIMD, δηλαδή εκτελούμε ένα πολύ απλό operation σε πολλά δεδομένα. Προσέχοντας τα μερικά αθροίσματα να γίνουν σε object-level και οι διαιρέσεις σε cluster-level (βλ. σημειώσεις υλοποίησης), μπορούμε να πραγματοποιήσουμε τον υπολογισμό αυτό σε O(1), μηδενίζοντας τον χρόνο εκτέλεσης σε CPU και διατηρώντας τον χρόνο σε GPU πρακτικά αμετάβλητο.

Τέλος, παρακάτω εμφανίζουμε τα διαγράμματα speedup για τα δύο configurations. Παρατηρούμε ξεκάθαρα την υπεροχή των μεθόδων Transpose/Shared σε σχέση με την Naive, ενώ στις περισσότερες περιπτώσεις η μέθοδος All-GPU πετυχαίνει σχεδόν το διπλάσιο speedup σε σχέση με τις προηγούμενες. Γενικά οι επιδόσεις είναι πολύ ικανοποιητικές, με την καλύτερη περίπτωση να οδηγεί σε επιτάχυνση 45x.




K-means - Speedup vs Block Size
Size:1024, Coords:2, Clusters:64, Loops:10
Naive
Transpose
Shared
All-GPU
50
40
30
20
10
0
speedup
32
48
64
128
238
512
1024
block size

4 Παραλληλοποίηση και βελτιστοποίηση αλγορίθμων σε αρχιτεκτονικές κατανεμημένης μνήμης

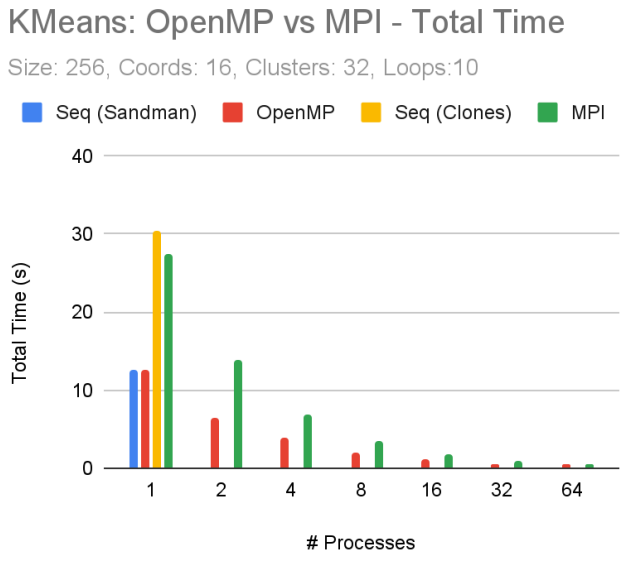
Σκοπός της τελευταίας άσκησης του εργαστηρίου είναι η εξοικείωση με το MPI, ένα πρωτόκολλο ανταλλαγής μηνυμάτων για συστήματα κατανεμημένης μνήμης. Μελετάμε τις επιδόσεις και την κλιμακωσιμότητα του MPI, μέσα από δύο εφαρμογές: η μία αφορά για άλλη μια φορά τον αλγόριθμο συσταδοποίησης K-Means, ενώ η άλλη την επίλυση της διαφορικής εξίσωσης διάδοσης θερμότητας σε δύο διαστάσεις.

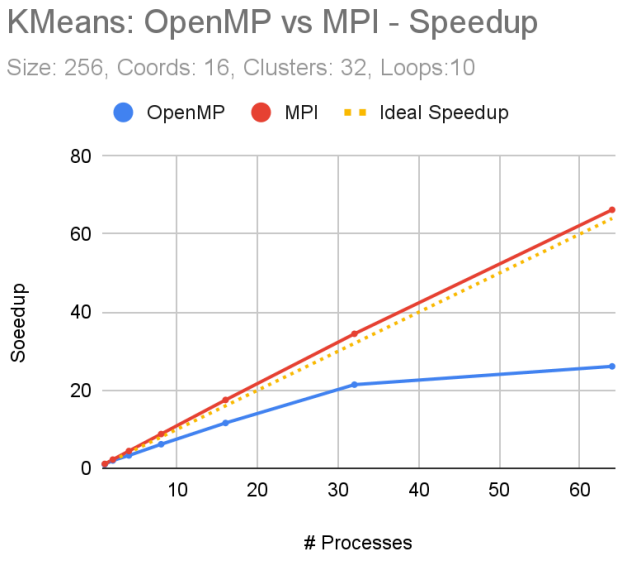
4.1 Άλλη μια παραλληλοποίηση και βελτιστοποίηση του αλγορίθμου K-Means

Ο πρώτος αλγόριθμος που υλοποιούμε με MPI είναι ο γνωστός K-Means. Μέσω αυτού του πειράματος σκοπεύουμε να συγκρίνουμε τις νέες μας υλοποιήσεις με μεθόδους που χρησιμοποιήσαμε νωρίτερα στο μάθημα, συγκεκριμένα με την OpenMP. Εκτελέσαμε τον MPI αλγόριθμο στη συστοιχία “clones”, που διαθέτει 8 κόμβους, με 8 πυρήνες ανά κόμβο. Συγκρίνουμε τις επιδόσεις μας με την καλύτερη υλοποίηση που πετύχαμε με OpenMP. Στον παρακάτω πίνακα φαίνονται οι λεπτομέρειες υλοποίησης και περιβάλλοντος εκτέλεσης των δύο υλοποιήσεων:

Μέθοδος	**MPI**	**OpenMP (Copy & Reduce)**
Συστοιχία	Clones	Sandman
Μνήμη	Κατανεμημένη	Κοινή
#Κόμβων	8	4
#Πυρήνων/Κόμβο	8	8 (16 με hyperthreading)
#Πυρήνων Συνολικά	64	32 (64 με hyperthreading)

Στα παρακάτω διαγράμματα εικονίζονται οι συνολικοί χρόνοι εκτέλεσης και οι επιταχύνσεις που επιτεύχθηκαν. Καθώς οι δύο υλοποιήσεις εκτελούνται σε διαφορετικά περιβάλλοντα εκτέλεσης, η ουσιαστική σύγκριση δε μπορεί να πραγματοποιηθεί με τους απόλυτους χρόνους εκτέλεσης, αλλά με το speedup. Για το λόγο αυτό, εκτελέστηκε ο ίδιος σειριακός αλγόριθμος σε κάθε συστοιχία, βάση του οποίου υπολογίζεται το αντίστοιχο speedup.





Οι επιδόσεις του MPI φαίνονται πάρα πολύ ικανοποιητικές. Εστιάζοντας στα διαγράμματα speedup, η OpenMP πετυχαίνει γραμμική αύξηση μέχρι τις 32 διεργασίες, με μια μειωμένη αύξηση στις 64 (λόγω hyperthreading). Από την άλλη μεριά, το speedup της υλοποίησης MPI ξεπερνάει το ιδανικό, με 66.2x επιτάχυνση για 64 πυρήνες! Αποδίδουμε αυτό το superlinear speedup στην χωρητικότητα των cache. Στο σειριακό πρόγραμμα, η είσοδος δε χωράει στην cache του ενός κόμβου, οπότε αναμένουμε μεγάλη καθυστέρηση λόγω προσβάσεων στη μνήμη. Από την άλλη μεριά, καθώς αυξάνουμε τους κόμβους, ο καθένας λαμβάνει από όλο και μικρότερο τμήμα των δεδομένων. Από ένα σημείο και μετά, τα δεδομένα χωράνε ολόκληρα στην cache του κάθε κόμβου, μηδενίζοντας τις προσβάσεις στη μνήμη. Έτσι, πέρα από το κέρδος σε υπολογιστική ισχύ, εξαλείφουμε και τις καθυστερήσεις λόγω πρόσβασης στη RAM. Έτσι πετυχαίνουμε speedup υψηλότερο από το θεωρητικό, που λαμβάνει υπόψη μόνο τον παράλληλο υπολογισμό.

Να σημειωθεί ότι η σύγκριση είναι “δίκαιη” μόνο μέχρι τις 32 διεργασίες, καθώς μετά στο OpenMP έχουμε μειωμένες επιδόσεις λόγω του hyperthreading, περιορισμού που οφείλεται στο υλικό. .Μπορούμε λοιπόν να αποφανθούμε ότι και οι δύο υλοποιήσεις πετυχαίνουν γραμμική επιτάχυνση, με την MPI να είναι αισθητά αποδοτικότερη και μάλιστα ιδανική.

4.2 Διάδοση θερμότητας σε δύο διαστάσεις

Ο επόμενος αλγόριθμος που θα προσπαθήσουμε να παραλληλοποιήσουμε είναι η μέθοδος Jacobi για τον υπολογισμό της μετάδοσης θερμότητας σε ένα δισδιάστατο χωρίο, από τα σύνορα προς το εσωτερικό. Συγκεκριμένα, σε κάθε χρονικό βήμα η τιμή κάθε κελιού υπολογίζεται από τον μέσο όρο των τεσσάρων γειτονικών του κελιών τη προηγούμενη χρονική στιγμή. Επομένως, ο υπολογισμός των

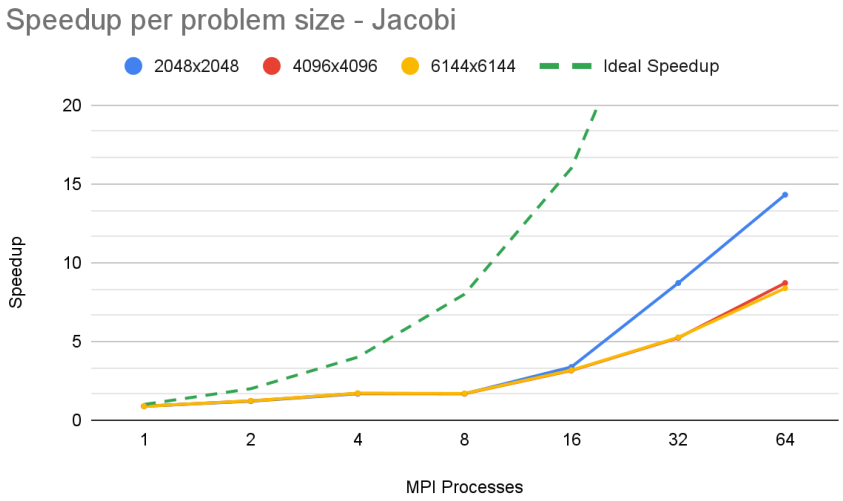
κελιών U[i][j] τη χρονική στιγμή t, είναι εντελώς ανεξάρτητος και κατά συνέπεια παράλληλος, πράγμα που θα εκμεταλλευτούμε στην υλοποίησή μας.

4.2.1 Μετρήσεις με έλεγχο σύγκλισης

Αρχικά πραγματοποιούμε μετρήσεις με έλεγχο σύγκλισης και μέγεθος πίνακα 512x512 τόσο για την σειριακή έκδοση της μεθόδου Jacobi όσο και για την παράλληλη με 64 MPI διεργασίες. Χρειάστηκαν 236000 επαναλήψεις και ο σειριακός χρόνος ήταν 397.7 sec, ενώ αυτός με MPI 59.7 sec, εκ των οποίων τα 4.1 sec ήταν χρόνος υπολογισμών και 1.3 sec χρόνος ελέγχου σύγκλισης, με τα υπόλοιπα 55.6 sec να είναι χρόνος που χρειάστηκε για την επικοινωνία και ανταλλαγή δεδομένων.

4.2.2 Μετρήσεις χωρίς έλεγχο σύγκλισης

Στη συνέχεια επαναλάβαμε το πείραμα χωρίς τον έλεγχο σύγκλισης, με σταθερό αριθμό επαναλήψεων t=256 και για μεγέθη πίνακα 2048x2048, 4096x4096, 6144x6144.



Το διάγραμμα speedup φανερώνει τη memory bandwidth bound φύση του προβλήματός μας. Μέχρι και τους 8 πυρήνες περιοριζόμαστε σε έναν υπολογιστικό κόμβο και άρα μία RAM. Αντιμετωπίζουμε λοιπόν το bottleneck του memory bus: ακόμα κι αν προσθέσουμε πυρήνες αυτοί απλά θα περιμένουν να προσπελάσουν τη μνήμη, με αποκορύφωμα την αύξηση από 4 σε 8 όπου δεν υπάρχει καμία βελτίωση. Από εκείνο το σημείο και μετά, διπλασιάζοντας του πυρήνες διπλασιάζουμε και τους κόμβους, άρα και τις μνήμες. Ξεκινάμε λοιπόν και βλέπουμε μια επιτάχυνση. Εδώ προκύπτει άλλος ένας διαχωρισμός. Στην περίπτωση του προβλήματος με μέγεθος 2048 έχουμε το επιπλέον πλεονέκτημα ότι τα τμήματα δεδομένων που λαμβάνουν οι κόμβοι είναι αρκετά μικρά ώστε να χωράνε στις cache των πυρήνων. Για το λόγο αυτό, το πρόβλημα μεγέθους 2048 έχει πολύ καλύτερη επιτάχυνση από τα 4096 και 6144 τα οποία, παρά τη διαφορά στο πλήθος υπολογισμών, οδηγούν σε

περίπου ίδια επιτάχυνση.

Στα διαγράμματα χρόνου φαίνεται η σταδιακή αύξηση του χρόνου επικοινωνίας των διεργασιών αυξάνοντας τις διεργασίες. Επιπλέον μεγαλώνοντας το μέγεθος του προβλήματος αυξάνεται ο χρόνος υπολογισμών με αποτέλεσμα η επικοινωνία να αποτελεί μικρότερο ποσοστό του συνολικού χρόνου.

